



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Titolo della Tesi

RELATORE

Prof. Nome relatore

Dott. Nome tutor

Università degli Studi di Salerno

CANDIDATO

Nome Cognome

Matricola: 0123456789

Anno Accademico YYYY-YYYY

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Dedica o citazione

Abstract

Il testo dell'abstract qui...

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Motivazioni e contesto	1
1.2 Obiettivi del progetto	2
1.3 Convenzioni linguistiche	3
1.4 Contributi progettuali e ingegneristici	3
1.5 Struttura del documento	4
2 Contesto e analisi del sistema	5
2.1 Il problema: frammentazione e inefficienza nella gestione dei contest	5
2.2 Stato dell'arte e analisi delle alternative	6
2.2.1 Strumenti generici adattati: lo "Status Quo"	7
2.2.2 Piattaforme per domini correlati	7
2.2.3 Posizionamento di Swift Contest	8
2.3 Attori e casi d'uso	8
2.3.1 Attori del sistema	8
2.3.2 Casi d'Uso: Gestione Account e Funzionalità Generali	9
2.3.3 Casi d'Uso: Gestione del Ciclo di Vita del contest	10

2.3.4	Casi d'Uso: Gestione delle Votazioni e dei Risultati	12
2.3.5	Casi d'Uso: Amministrazione del Sistema	12
2.4	Requisiti del sistema	13
2.4.1	Requisiti funzionali (Functionality)	13
2.4.2	Requisiti non funzionali	15
2.5	Modello del dominio	16
2.5.1	Entità fondamentali	18
2.5.2	Vincoli chiave del dominio	18
3	Tecnologie e strumenti utilizzati	20
3.1	Stack tecnologico principale	20
3.1.1	Frontend: Flutter e Dart	20
3.1.2	Backend: Supabase, PL/pgSQL e TypeScript	21
3.2	Piattaforme, Strumenti e Servizi	22
3.2.1	Piattaforme di Deployment e Servizi Cloud	22
3.2.2	Strumenti di Sviluppo e Amministrazione	22
3.2.3	Version Control e Code Hosting	23
3.2.4	Strumenti di Documentazione e Modellazione	24
4	Progettazione e implementazione del sistema	25
4.1	Linee guida progettuali	25
4.2	Architettura del frontend: il pattern MVVM con BLoC	26
4.2.1	Implementazione del ViewModel: il pattern BLoC	27
4.2.2	Navigazione con AutoRoute	31
4.2.3	Strategia di navigazione e passaggio dei dati	32
4.2.4	Design system e theming	33
4.3	Progettazione del database	36
4.3.1	Schema fisico e tabelle principali	37
4.3.2	Scelte di design chiave	38
4.3.3	Implicazioni del design del form di voto sull'aggregazione dei risultati	41
4.4	Architettura del backend	42
4.4.1	Funzioni RPC (Remote Procedure Call)	43

4.4.2	Edge Functions	45
4.5	Architettura di Sicurezza: Principio del Minimo Privilegio e Accesso Mediato	47
4.5.1	Strategia di Implementazione: Centralizzazione della Logica di Autorizzazione	47
4.5.2	Il Ruolo della RLS e dell'API come "Gatekeeper"	47
4.5.3	L'Eccezione Necessaria: le Policy RLS per il Servizio Realtime	48
4.5.4	Misure di Sicurezza Aggiuntive	48
4.6	Progettazione e Implementazione degli Strumenti di Amministrazione	51
4.6.1	Esigenza di un Pannello di Amministrazione	51
4.6.2	Scelta di Retool per lo Sviluppo Rapido	51
4.6.3	Architettura di Comunicazione Sicura tra Retool e Supabase .	52
4.6.4	Funzionalità Implementate nel Pannello	55
5	Conclusioni	56
5.1	Sintesi del lavoro svolto e risultati ottenuti	56
5.2	Limiti e analisi critica	57
5.3	Prospettive e sviluppi futuri	58
5.4	Considerazioni finali	59

Bibliografia

Elenco delle figure

2.1	Confronto tra il flusso di lavoro tradizionale e l'approccio integrato di Swift Contest.	6
2.2	Diagramma dei Casi d'Uso: Gestione Account e Funzionalità Generali.	10
2.3	Diagramma dei Casi d'Uso: Gestione del Ciclo di Vita del contest.	11
2.4	Diagramma dei Casi d'Uso: Gestione delle Votazioni e dei Risultati.	12
2.5	Diagramma dei Casi d'Uso: Amministrazione del Sistema.	13
2.6	Diagramma delle classi	17
4.1	Architettura Client-Server del sistema Swift Contest con backend BaaS.	26
4.2	Architettura del Client MVVM con BLoC.	27
4.3	Diagramma Entità-Relazioni (ERD) dello schema del database.	37
4.4		52
4.5		54

Elenco delle tabelle

4.1	Elenco esemplificativo delle Funzioni RPC del sistema.	44
4.2	Elenco esemplificativo delle Edge Functions del sistema.	45

CAPITOLO 1

Introduzione

1.1 Motivazioni e contesto

La gestione di *contest* in tempo reale rappresenta a oggi un processo frammentato e complesso. Organizzatori, partecipanti e giurati spesso ricorrono a strumenti diversi: servizi di posta elettronica per la comunicazione degli inviti, piattaforme di file sharing per la raccolta dei lavori, fogli di calcolo per la tabulazione dei punteggi.

Questa frammentazione genera inefficienze operative, rischi di errore umano e una significativa difficoltà nel garantire trasparenza e sicurezza. L'analisi dello stato dell'arte, come verrà dettagliato nel capitolo successivo, ha inoltre rivelato un **significativo vuoto di mercato**: non esistono soluzioni verticali e accessibili che coprano in modo integrato l'intero ciclo di vita di un *contest*.

Il progetto **Swift Contest** nasce proprio per colmare questa lacuna, proponendo un sistema moderno, sicuro e multiplatforma che unisce in un'unica applicazione le principali fasi del ciclo di vita di un *contest*: dalla creazione e configurazione, all'invito dei partecipanti e dei giurati, fino alla raccolta dei lavori e alla gestione delle sessioni di voto in tempo reale.

1.2 Obiettivi del progetto

L'obiettivo primario di questo lavoro è stato quello di **progettare e implementare l'architettura** di un sistema completo di *contest management*, adottando tecnologie moderne e approcci architetturali rigorosi.

Gli obiettivi specifici, che hanno guidato la progettazione, possono essere così sintetizzati:

- **Sviluppo cross-platform:** garantire un'esperienza utente fluida e unificata su più piattaforme (mobile e web) attraverso lo sviluppo di un client basato su un'unica codebase.
- **Sicurezza "by design":** adottare un'architettura di sicurezza basata sul **Principio del Minimo Privilegio (Principle of Least Privilege)**, dove l'accesso al database è negato per default e concesso selettivamente solo tramite un'API controllata.
- **Esperienza utente mirata:** progettare interfacce utente intuitive e funzionali, modellate sui ruoli specifici dei diversi attori del sistema (organizzatore, partecipante, giurato).
- **Accesso flessibile alla votazione:** implementare un sistema di voto che supporti due modalità di accesso per i giurati. Oltre ai giurati nominati (utenti registrati e invitati in giurie di tipo *nominato (appointed)*), il sistema deve permettere la partecipazione di **giurati semplici**. Questo avviene tramite la creazione, da parte dell'organizzatore, di una **giuria di tipo semplice (simple)** a cui è associato un token di accesso sicuro, abbassando così la barriera d'ingresso per i giurati occasionali (siano essi ospiti o utenti già registrati).
- **Affidabilità e auditabilità del processo:** assicurare l'integrità e la verificabilità del processo di voto attraverso meccanismi tecnici robusti, come lo *snapshot* dei dati al momento dell'avvio della sessione e la possibilità di applicare restrizioni basate sulla geolocalizzazione.

1.3 Convenzioni linguistiche

Al fine di garantire la massima chiarezza e coerenza con gli standard del settore, nel presente elaborato è stata adottata una convenzione linguistica specifica per gli elementi prettamente tecnici.

Per tutti gli artefatti di progettazione e implementazione, come i **diagrammi UML**, le **figure architetturali**, i **listati di codice** e i nomi delle componenti software, è stato scelto di utilizzare la **lingua inglese**. Questa decisione riflette lo standard de facto della comunità informatica internazionale, assicurando che la documentazione tecnica sia immediatamente comprensibile e allineata alle convenzioni del settore. Nel testo descrittivo, termini italiani e inglesi potranno essere usati come sinonimi dove appropriato (es. contest/concorso).

1.4 Contributi progettuali e ingegneristici

Il valore principale di questo lavoro di tesi risiede nell'applicazione rigorosa di principi di ingegneria del software moderni per risolvere un problema pratico. Il contributo non è di natura teorica, ma **progettuale e architetturale**.

Il progetto **Swift Contest** si presenta come un *blueprint*, ovvero un modello di riferimento, per la creazione di applicazioni collaborative complesse e sicure. I contributi specifici sono:

- **Proposta di una nuova categoria di prodotto:** Data l'assenza di soluzioni verticali, il progetto stesso si configura come un caso di studio per una nuova categoria di software dedicato alla gestione dei *contest*, colmando un vuoto di mercato.
- **Dimostrazione di un'architettura sicura su Backend-as-a-Service (BaaS):** Il sistema mostra come implementare un'architettura basata sul Principio del Minimo Privilegio utilizzando una piattaforma Backend-as-a-Service (BaaS).
- **Modello Dati robusto per processi immutabili:** Viene presentato un design del database che garantisce l'integrità e l'attendibilità dei processi di voto tramite l'uso di "attributi di snapshot" e denormalizzazione controllata.

In sintesi, il valore della tesi è nel dimostrare come, partendo da un'esigenza reale, sia stato possibile progettare e realizzare una soluzione completa, sicura e ben ingegnerizzata, utilizzando strumenti accessibili e seguendo le best practice del settore.

1.5 Struttura del documento

Il presente elaborato è organizzato in sei capitoli principali, che guidano il lettore attraverso le fasi di analisi, selezione tecnologica, progettazione, implementazione e valutazione del progetto.

- **Capitolo 1 – Introduzione:** illustra le motivazioni, gli obiettivi di alto livello e il valore del lavoro di tesi.
- **Capitolo 2 – Contesto e analisi del sistema:** definisce in dettaglio il problema, analizza lo stato dell'arte e presenta i requisiti del sistema.
- **Capitolo 3 – Tecnologie e strumenti utilizzati:** descrive lo stack tecnologico adottato, giustificando le scelte chiave in relazione agli obiettivi del progetto e ai vincoli di sviluppo.
- **Capitolo 4 – Progettazione e implementazione del sistema:** descrive l'architettura software, il design pattern adottato, la struttura del database e l'implementazione delle principali funzionalità, con esempi di codice e diagrammi tecnici.
- **Capitolo 5 – Distribuzione e prodotto finale:** presenta il processo di deploy, l'interfaccia utente e i flussi di utilizzo reali.
- **Capitolo 6 – Conclusioni:** riflette sui risultati ottenuti, ne analizza i limiti e propone possibili sviluppi futuri.

Contesto e analisi del sistema

2.1 Il problema: frammentazione e inefficienza nella gestione dei contest

La gestione di un contest, specialmente in ambiti creativi, è un processo che coinvolge una molteplicità di attori (organizzatori, partecipanti, giurati) e fasi operative sequenziali e dipendenti. L'approccio tradizionale si affida a un assemblaggio di strumenti generici e non integrati, creando un flusso di lavoro discontinuo, inefficiente e soggetto a errori.

Le criticità di questo modello possono essere analizzate nel dettaglio:

- **Frammentazione del processo e dei dati:** la comunicazione degli inviti avviene tramite servizi di posta elettronica, la sottomissione dei lavori tramite posta elettronica o piattaforme di cloud storage (es. Google Drive) e la tabulazione dei voti tramite fogli di calcolo (es. Microsoft Excel). Questo approccio disperde le informazioni, rende difficile avere una visione d'insieme e richiede un costante lavoro manuale di organizzazione dei dati.
- **Mancanza di integrità e consistenza dei dati:** la gestione manuale aumenta esponenzialmente il rischio di errori. Esempi comuni includono la perdita di

iscrizioni, l’errata trascrizione dei punteggi o l’inclusione di partecipanti non idonei. Non esiste un’unica “fonte di verità” per i dati del contest.

- **Carenza di strumenti in tempo reale:** l’assenza di un sistema centralizzato impedisce di gestire sessioni di voto live in modo coordinato e trasparente.
- **Rischi di sicurezza e potenziale di abuso:** la sottomissione di voti multipli, la partecipazione di utenti non invitati e la gestione di conflitti di interesse diventano problemi concreti e di difficile gestione.

L’obiettivo primario del sistema **Swift Contest** è superare queste criticità strutturali, fornendo un ambiente end-to-end integrato, sicuro e automatizzato, che funga da unica fonte di verità per l’intero ciclo di vita del contest.

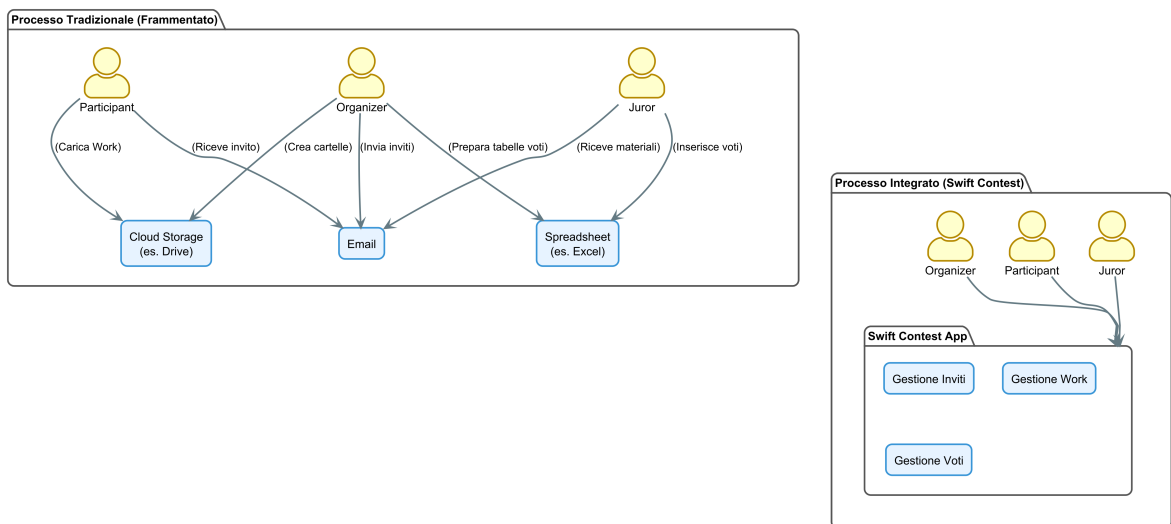


Figura 2.1: Confronto tra il flusso di lavoro tradizionale e l’approccio integrato di Swift Contest.

2.2 Stato dell’arte e analisi delle alternative

L’analisi dello stato dell’arte ha rivelato un’interessante lacuna nel panorama software attuale: **non esistono soluzioni verticali e accessibili che coprano in modo integrato l’intero ciclo di vita di un contest**. Le pratiche correnti si affidano a due approcci principali, entrambi inadeguati.

2.2.1 Strumenti generici adattati: lo “Status Quo”

Questa è la pratica più diffusa e rappresenta il problema fondamentale che **Swift Contest** si propone di risolvere. Consiste nell'utilizzare una combinazione di strumenti generici, non pensati per questo scopo, creando un flusso di lavoro frammentato e di difficile gestione, ad esempio:

- **Google Forms** per la raccolta delle iscrizioni e la sottomissione dei voti.
- **Google Drive** per la sottomissione dei lavori.
- **Google Sheets** per la tabulazione manuale dei voti.
- **Email** per tutte le comunicazioni.

Come già analizzato, questo approccio soffre di gravi criticità in termini di sicurezza, integrità dei dati ed efficienza operativa, che hanno motivato la nascita del progetto.

2.2.2 Piattaforme per domini correlati

Sono state analizzate anche piattaforme software specializzate in domini vicini a quello dei contest, per verificare la possibilità di un loro adattamento. Tuttavia, nessuna di esse si è rivelata una soluzione praticabile.

Piattaforme di e-voting

Sistemi come *Helios Voting* sono progettati per il **voto politico o assembleare**. Il loro focus è sulla crittografia e sull'anonimato del votante. Di conseguenza, mancano completamente delle funzionalità necessarie per un contest, quali:

- La possibilità di invitare partecipanti e creare giurie.
- La sottomissione e gestione di opere digitali.
- La creazione di form di voto personalizzati con campi testuali e numerici.

Il loro scopo è intrinsecamente diverso e non sono in alcun modo utilizzabili.

Piattaforme per la gestione di eventi

Soluzioni commerciali come *Eventbrite* sono focalizzate sulla **gestione logistica di eventi e sulla vendita di biglietti**. Sebbene eccellano nella gestione delle registrazioni, non offrono alcuna funzionalità specifica per il dominio dei contest, come:

- La sottomissione di opere digitali.
- La creazione di giurie con ruoli definiti.
- La configurazione di form di valutazione strutturati.
- La gestione di sessioni di voto in tempo reale.

Il loro scopo è completamente diverso e non risultano realisticamente adattabili ai requisiti di un contest.

2.2.3 Posizionamento di Swift Contest

L'analisi conferma che Swift Contest si posiziona in un **vuoto di mercato e tecnologico**. Non mira a competere con le piattaforme di e-voting o di gestione eventi, ma a creare una **nuova categoria di prodotto**: una soluzione verticale, integrata e accessibile, progettata specificamente per le esigenze di organizzatori, partecipanti e giurati di un contest.

2.3 Attori e casi d'uso

Il sistema è progettato per servire diversi tipi di utenti, ciascuno con un ruolo e un insieme di permessi ben definiti. L'interazione tra questi attori e le funzionalità del sistema è stata modellata attraverso una serie di diagrammi dei Casi d'Uso, suddivisi in pacchetti logici (packages) che legano funzionalità della stessa categoria per migliorarne leggibilità e modularità.

2.3.1 Attori del sistema

L'analisi del dominio ha permesso di identificare i seguenti attori principali:

- **Guest:** rappresenta un utente non autenticato. In questo sistema, che non prevede contenuti pubblici, un *Guest* non ha privilegi di visualizzazione. Le sue uniche capacità sono quelle di **autenticarsi** (tramite registrazione o login) o di agire come **Simple Juror** se in possesso di un token di accesso.
- **Authenticated:** è un attore astratto che rappresenta un utente che ha completato l'autenticazione. Funge da base per i ruoli operativi, che ne ereditano le funzionalità comuni come la gestione del proprio profilo e delle notifiche.
- **Organizer:** è un utente autenticato responsabile della creazione e della gestione completa di un contest.
- **Participant:** è un utente autenticato che si iscrive a un contest per sottoporre il proprio lavoro.
- **Juror:** è un utente autenticato il cui ruolo primario è quello di far parte di una giuria nominata (*appointed jury*), oppure votare come *Simple Juror* se in possesso di un token per la votazione.
- **Admin:** è un ruolo di amministrazione, gestito tramite una dashboard esterna (Retool), con privilegi elevati per la manutenzione del sistema.

2.3.2 Casi d'Uso: Gestione Account e Funzionalità Generali

Questa area copre le funzionalità di base legate all'accesso e alla gestione dell'account utente. Include i processi di autenticazione, la modifica del profilo personale e la gestione delle notifiche interne.

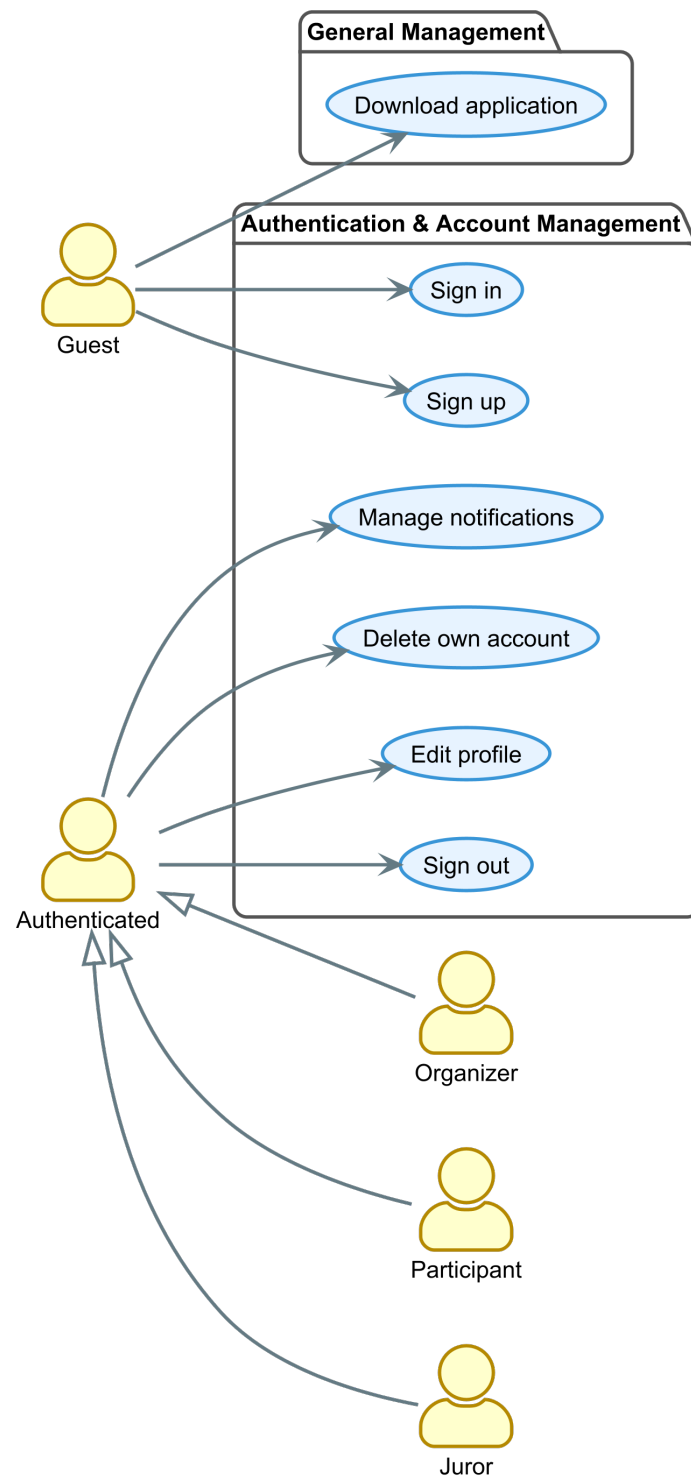


Figura 2.2: Diagramma dei Casi d'Uso: Gestione Account e Funzionalità Generali.

2.3.3 Casi d'Uso: Gestione del Ciclo di Vita del contest

Questa è l'area funzionale più vasta e descrive il cuore del processo organizzativo. Copre la creazione di un contest, la gestione dei suoi membri (partecipanti e giurati)

e la sottomissione dei lavori. Gli attori principali in questo contesto sono l'*Organizer* e il *Participant*.

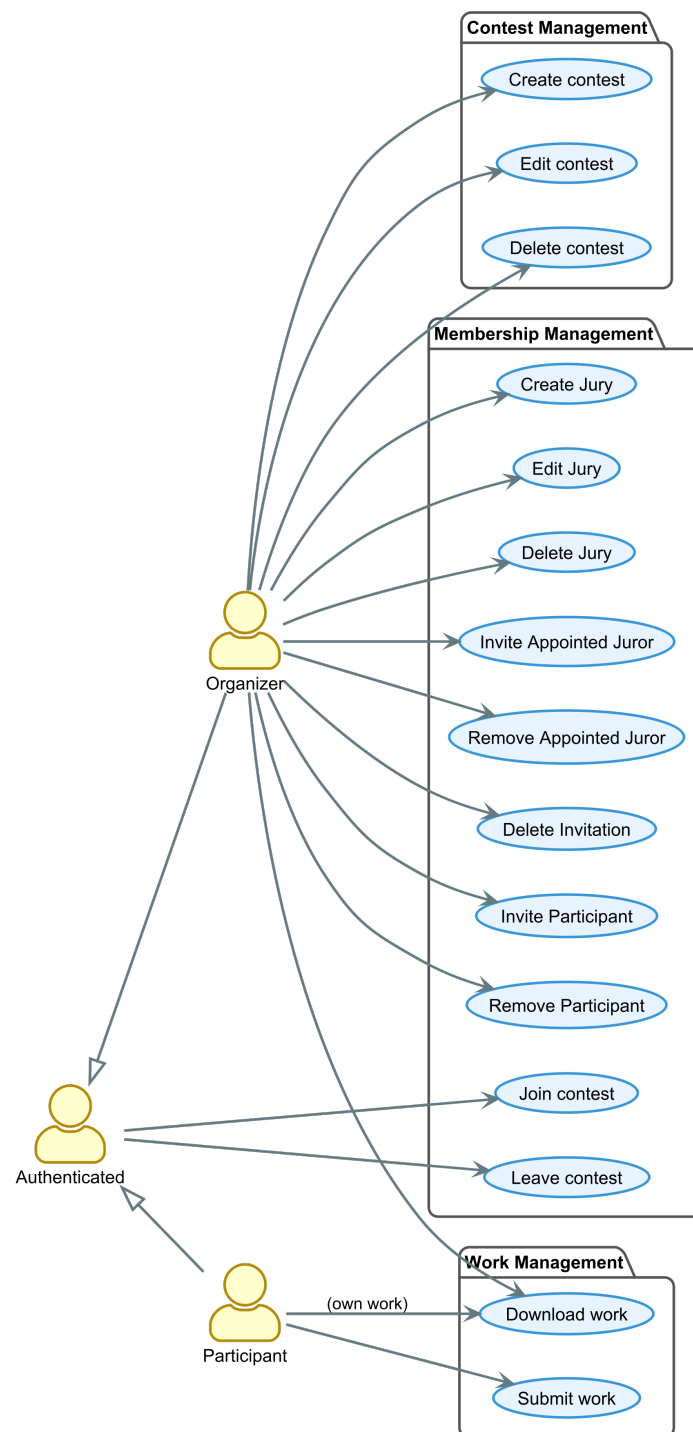


Figura 2.3: Diagramma dei Casi d'Uso: Gestione del Ciclo di Vita del contest.

2.3.4 Casi d'Uso: Gestione delle Votazioni e dei Risultati

Questa sezione si concentra interamente sul processo critico della votazione. Include la configurazione del form di voto, la conduzione della sessione di voto (anche con restrizioni di geolocalizzazione) e la generazione e pubblicazione delle classifiche. Gli attori principali qui sono l'*Organizer* e l'*Juror*.

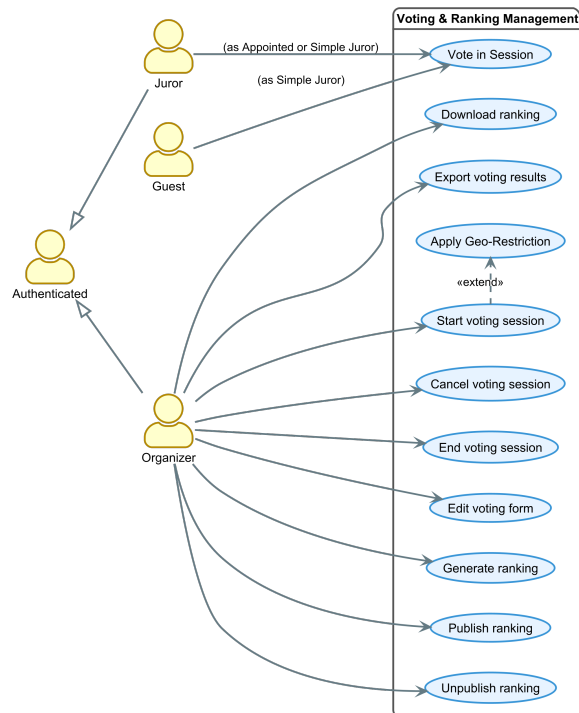


Figura 2.4: Diagramma dei Casi d'Uso: Gestione delle Votazioni e dei Risultati.

2.3.5 Casi d'Uso: Amministrazione del Sistema

Quest'ultima area isola le funzionalità di amministrazione, che non sono accessibili tramite l'applicazione principale ma attraverso una dashboard esterna. Queste operazioni sono riservate all'attore *Admin* e sono fondamentali per la manutenzione e la moderazione della piattaforma.

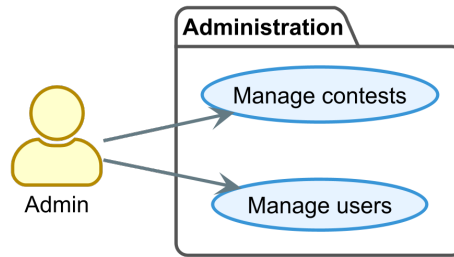


Figura 2.5: Diagramma dei Casi d'Uso: Amministrazione del Sistema.

2.4 Requisiti del sistema

I requisiti del sistema sono stati classificati secondo il modello **FURPS+**, un'estensione del modello FURPS che include categorie aggiuntive per coprire in modo esaustivo tutti i vincoli di un progetto software.

2.4.1 Requisiti funzionali (Functionality)

- **RF-01: Gestione Account Utente:** Il sistema deve permettere a un utente di registrarsi (*Sign Up*) e autenticarsi (*Sign In*).
- **RF-02: Gestione del Profilo Personale:** Ogni utente autenticato deve poter visualizzare e modificare i dati del proprio profilo (es. nome completo). Deve inoltre essere garantita la possibilità di eliminare il proprio account in modo autonomo (*self-service deletion*).
- **RF-03: Gestione Contest:** Il sistema deve permettere a un *Organizer* di creare, modificare e cancellare i propri contest.
- **RF-04: Gestione Giurie e Form di Voto:** Il sistema deve permettere a un *Organizer* di creare giurie (di tipo *appointed* o *simple*) e di configurare un *VotingForm* associato, con campi organizzati secondo il loro **scope** e tipo.
- **RF-05: Gestione Iscrizioni:** Il sistema deve permettere a un *Organizer* di invitare partecipanti e giurati nominati tramite email con token univoci.

- **RF-06: Gestione Esclusioni di Voto:** Il sistema deve permettere a un *Organizer* di configurare delle esclusioni, impedendo a specifici *Appointed Juror* di valutare determinati *Participant* all'interno di una sessione di voto.
- **RF-07: Accettazione Inviti e Accesso al contest:** Il sistema deve permettere a un utente di utilizzare un token di invito per formalizzare la propria iscrizione a un contest, sia come *Participant* che come *Appointed Juror*.
- **RF-08: Sottomissione Lavori:** Il sistema deve permettere a un *Participant* di sottomettere un proprio *Work* (Lavoro Digitale) per un contest a cui è iscritto.
- **RF-09: Restrizione Geografica della Votazione:** Il sistema deve permettere a un *Organizer* di imporre una restrizione geografica a una sessione di voto, richiedendo ai giurati di trovarsi entro un raggio specifico da un luogo predefinito.
- **RF-10: Processo di Votazione:** Il sistema deve permettere a un utente autorizzato (sia *Appointed Juror* che *Simple Juror*) di inviare i propri voti durante una sessione di voto attiva, rispettando le eventuali esclusioni configurate.
- **RF-11: Gestione Classifiche e Risultati:** Il sistema deve permettere all'*Organizer* di generare una classifica per ogni singola giuria, di esportare i risultati grezzi in formato Excel e di pubblicare una classifica finale in formato PDF.
- **RF-12: Visualizzazione dei Risultati:** Il sistema deve permettere a *Participant* e *Juror* di un contest di visualizzare e scaricare le classifiche finali pubblicate dall'organizzatore.
- **RF-13: Notifiche Inbox:** Il sistema deve fornire a ogni utente autenticato una inbox per ricevere notifiche generate automaticamente da eventi di sistema.
- **RF-14: Amministrazione del Sistema:** Il sistema deve esporre un'interfaccia sicura per permettere a un *Admin* di eseguire operazioni di alto livello, come la visualizzazione di tutti gli utenti o la rimozione forzata di un contest.

2.4.2 Requisiti non funzionali

- **Usabilità (Usability):** L'interfaccia utente deve essere intuitiva, facile da apprendere e coerente tra la versione mobile e quella web.
- **Affidabilità (Reliability):** Il sistema deve essere robusto e garantire l'integrità dei dati. La logica di *snapshot* delle sessioni di voto è un requisito chiave per assicurare che il processo di valutazione sia immune da modifiche ai dati.
- **Performance:** L'applicazione deve essere reattiva, con tempi di caricamento inferiori ai 3 secondi.
- **Manutenibilità (Supportability):** L'architettura software deve essere modulare. L'adozione del pattern MVVM/BLoC per il frontend garantisce una netta separazione delle responsabilità, facilitando futuri interventi.
- **Implementazione (Implementation):**
 - Il client deve essere sviluppato in **Dart** con il framework **Flutter** per le piattaforme Android e Web.
 - Il backend deve basarsi sulla piattaforma **Supabase**, utilizzando **PostgreSQL**, **PL/pgSQL** per le RPC e **TypeScript** per le Edge Functions.
 - La gestione dello stato nel client deve seguire il pattern **BLoC**.
- **Interfaccia (Interface):**
 - Il sistema deve interfacciarsi in modo sicuro con servizi esterni come **Resend** (per le email) e **Google Places** (per la geolocalizzazione).
 - Deve essere prevista un'API sicura per l'interfacciamento con strumenti di amministrazione esterni (**Retool**).
- **Operativi (Operation):**
 - Il deployment del backend deve essere tracciabile tramite migrazioni SQL gestite dalla CLI di Supabase.
 - Il deployment del frontend web deve essere automatizzato tramite una pipeline di CI/CD.

- I segreti (API keys) devono essere gestiti in modo sicuro, senza essere esposti nel codice sorgente.
- **Distribuzione (Packaging):**
 - L'applicazione web deve essere accessibile tramite URL pubblico.
 - L'applicazione Android deve essere distribuita come pacchetto **APK**, tramite un meccanismo di download sicuro.
- **Legali (Legal):**
 - Il sistema deve essere sviluppato utilizzando principalmente tecnologie con licenze open-source permissive.
 - La gestione dei dati personali deve essere orientata ai principi del GDPR, che garantisce all'utente il diritto alla cancellazione.

2.5 Modello del dominio

Il dominio applicativo è stato modellato a partire dalle entità fondamentali e dalle loro relazioni. Il Diagramma di Classe Concettuale (UML) è astratto dai dettagli implementativi e si focalizza sui concetti di business e sui vincoli che li governano.

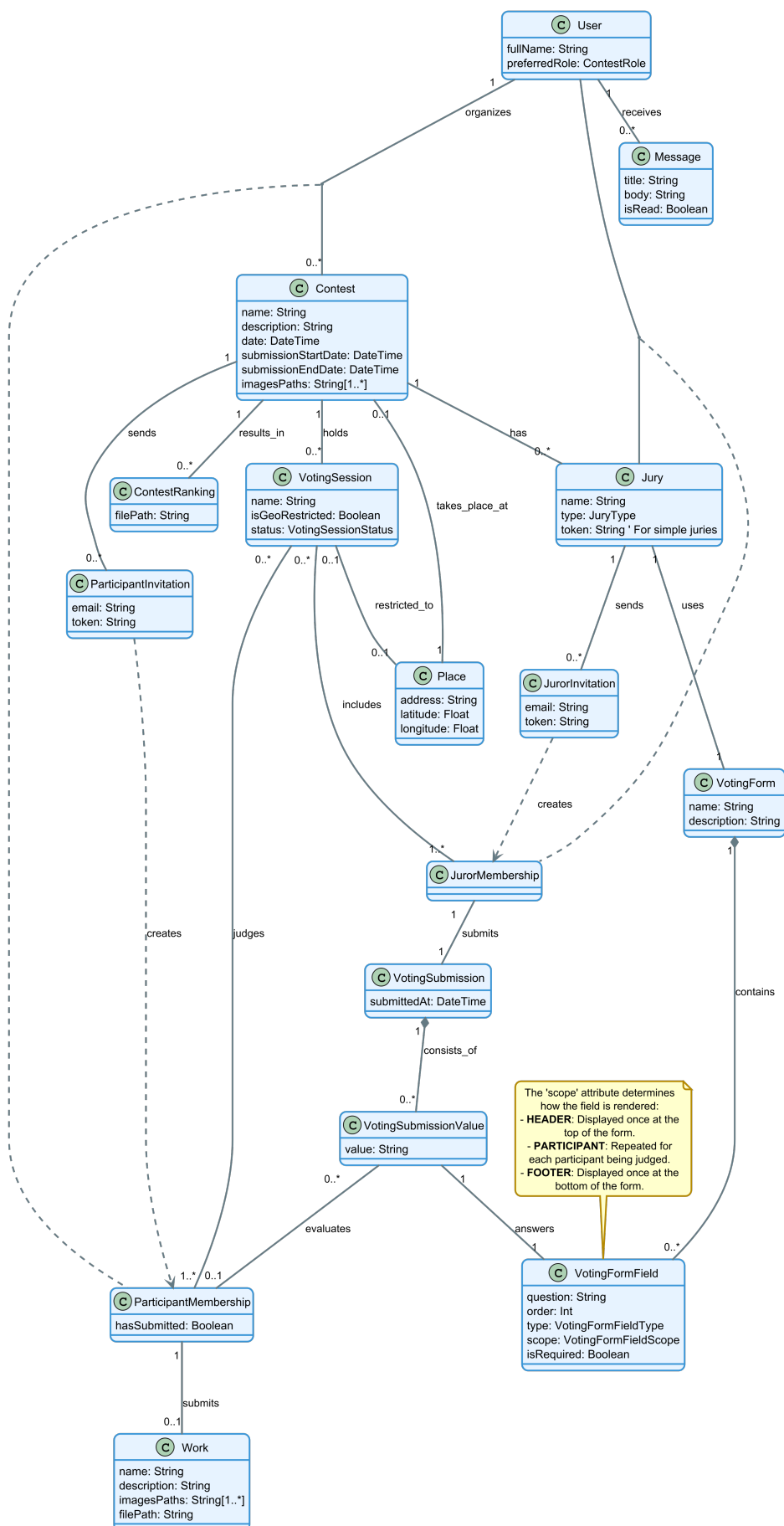


Figura 2.6: Diagramma delle classi

2.5.1 Entità fondamentali

- **User:** l'utente del sistema, contenente informazioni del profilo.
- **Contest:** l'entità centrale, che aggrega tutte le informazioni relative a un concorso.
- **ParticipantMembership** e **JurorMembership:** classi associative che risolvono le relazioni "multi-a-molti" tra utenti e contest/giurie.
- **Work:** l'elaborato unico caricato da un partecipante. È legato a una *ParticipantMembership*.
- **Jury:** una giuria all'interno di un contest, che può essere di tipo *appointed* o *simple*.
- **VotingForm** e **VotingFormField:** la struttura del form di votazione. L'attributo *scope* del campo ne determina il comportamento.
- **VotingSession:** una sessione di voto live. Al suo avvio, il sistema "congela" lo stato del contest creando delle righe dedicate in tabelle di sessione (es. `voting_session_participants`), che contengono non solo i riferimenti alle entità originali, ma anche una copia di **attributi di snapshot** (es. il nome del partecipante, il nome dell'opera) per garantire l'immutabilità del processo.
- **ContestRanking:** la classifica finale, rappresentata come un riferimento a un file nello storage.
- **Message:** una notifica di sistema inviata a un utente.

2.5.2 Vincoli chiave del dominio

Il modello dati è stato progettato per far rispettare programmaticamente alcune regole di business fondamentali:

1. **Unicità del Work:** un partecipante può sottoporre un solo *Work* per ogni concorso a cui è iscritto, garantito da un vincolo di unicità a livello di database.

2. **Calcolo della Classifica Semplice:** la classifica viene generata esclusivamente sulla base dei campi di tipo *slider* con scope *participant*. I campi testuali non contribuiscono al punteggio.
3. **Immutabilità della Votazione:** il meccanismo di snapshot garantisce che una volta avviata una sessione di voto, i dati rilevanti dei partecipanti, dei giurati e la struttura del form vengano “congelati”. Questo avviene tramite la copia di attributi chiave in tabelle di sessione dedicate, rendendo il processo di valutazione immune da modifiche successive ai dati originali.
4. **Accesso Ospite Controllato:** l’accesso come *Simple Juror* è strettamente controllato da un token associato a una specifica giuria di tipo semplice (simple jury).

Tecnologie e strumenti utilizzati

La realizzazione del progetto **Swift Contest** è stata resa possibile dall'adozione di un insieme di tecnologie, piattaforme e strumenti moderni, selezionati per soddisfare i requisiti di cross-platform, sicurezza, scalabilità e velocità di sviluppo. Questo capitolo descrive lo stack tecnologico utilizzato, giustificando le scelte chiave in relazione agli obiettivi del progetto.

3.1 Stack tecnologico principale

Questa sezione descrive i linguaggi e i framework utilizzati per la scrittura del codice sorgente dell'applicazione, sia lato client che lato server.

3.1.1 Frontend: Flutter e Dart

- **Flutter:** È il framework UI open-source di Google scelto per lo sviluppo dell'applicazione client. La sua caratteristica principale è la capacità di generare applicazioni compilate nativamente per mobile, web e desktop da un'unica codebase. Questa scelta è stata fondamentale per soddisfare il requisito di **svi-**

luppo cross-platform, garantendo un'esperienza utente coerente e riducendo drasticamente i tempi e i costi di sviluppo e manutenzione.

- **Dart:** È il linguaggio di programmazione su cui si basa Flutter. È un linguaggio moderno, orientato agli oggetti e fortemente tipizzato, ottimizzato per la creazione di interfacce utente. La sua architettura, che supporta sia la compilazione Just-In-Time (JIT) per uno sviluppo rapido (con hot reload) sia la compilazione Ahead-Of-Time (AOT) per performance elevate in produzione, lo ha reso la scelta naturale per questo progetto.

3.1.2 Backend: Supabase, PL/pgSQL e TypeScript

- **Supabase:** È la piattaforma open-source **Backend-as-a-Service (BaaS)** che costituisce il cuore dell'infrastruttura server. È stata scelta perché offre, in un'unica soluzione integrata, tutti i servizi necessari: un database **PostgreSQL** completo, un sistema di autenticazione, uno storage per i file, API auto-generate e servizi serverless (RPC ed Edge Functions).
- **PL/pgSQL:** È il linguaggio procedurale di PostgreSQL. È stato utilizzato per scrivere tutte le **Funzioni RPC (Remote Procedure Call)** del sistema. La scelta di implementare la logica di accesso ai dati direttamente a livello di database tramite questo linguaggio ha permesso di ottenere performance elevate e di garantire l'atomicità delle operazioni, sfruttando al contempo il potente sistema di sicurezza di PostgreSQL.
- **TypeScript:** È stato scelto come linguaggio per lo sviluppo delle **Edge Functions**, eseguite sul runtime Deno di Supabase. La sua natura fortemente tipizzata, unita alla vasta disponibilità di librerie dell'ecosistema JavaScript/TypeScript, lo ha reso ideale per gestire logiche complesse, transazioni multi-step e, soprattutto, per orchestrare le interazioni sicure con le API di servizi esterni.

3.2 Piattaforme, Strumenti e Servizi

Questa sezione descrive le piattaforme di terze parti e gli strumenti che hanno supportato il ciclo di vita del progetto, dal development al deployment, fino alla documentazione.

3.2.1 Piattaforme di Deployment e Servizi Cloud

- **Vercel:** È la piattaforma serverless utilizzata per il deployment e l'hosting della versione **Web** dell'applicazione Flutter. La scelta è ricaduta su Vercel principalmente per il suo **piano gratuito**, che ha permesso di distribuire l'applicazione senza sostenere costi. Oltre a questo, Vercel è stato preferito per le sue caratteristiche tecniche di alto livello, come l'integrazione nativa con i framework frontend moderni e una pipeline di CI/CD completamente automatizzata.
- **Resend:** È il servizio transazionale di invio **email** utilizzato dal sistema. È stato integrato tramite la propria API REST, invocata da una Edge Function, per gestire l'invio di tutte le email automatiche, come gli inviti per partecipanti e giurati.
- **Google Cloud:** Sebbene il progetto non sia ospitato su Google Cloud, è stato utilizzato uno dei suoi servizi API: la **Google Places API**. Questo servizio è stato integrato per fornire la funzionalità di ricerca e autocompletamento degli indirizzi nella creazione dei contest e nella configurazione delle sessioni di voto per la geolocalizzazione.

3.2.2 Strumenti di Sviluppo e Amministrazione

- **IntelliJ IDEA:** È l'Integrated Development Environment (IDE) di JetBrains utilizzato per la scrittura del codice. Sebbene lo sviluppo iniziale fosse stato avviato su **Android Studio**, si è scelto di migrare a **IntelliJ IDEA Community Edition**. Questa decisione è stata motivata da una maggiore stabilità, una minore incidenza di bug e una superiore reattività dell'IDE, specialmente nel caricamento iniziale del progetto. Grazie alla possibilità di integrare il supporto

completo per lo sviluppo in Flutter e Dart tramite plugin ufficiali, IntelliJ IDEA ha offerto un ambiente di sviluppo più performante e affidabile.

- **Material Theme Builder:** È uno strumento ufficiale di Google (disponibile online e su Figma) utilizzato per generare la base del tema Material Design 3. A partire da un singolo colore “seme” (seed color), lo strumento genera automaticamente l’intera palette di colori (ColorScheme) per il tema chiaro e scuro, garantendo coerenza cromatica e accessibilità. Il codice Dart generato da questo strumento è stato poi utilizzato come base per il `MaterialTheme` dell’applicazione.
- **Node Package Manager (npm):** È il gestore di pacchetti dell’ecosistema JavaScript/Node.js. Sebbene il progetto non sia basato su Node.js, npm è stato uno strumento indispensabile per l’installazione e la gestione di dipendenze di sviluppo, prima tra tutte la **CLI di Supabase**, utilizzata per gestire le migrazioni del database e il deployment delle Edge Functions.
- **Retool:** È la piattaforma low-code utilizzata per costruire rapidamente la **dashboard di amministrazione** interna. La sua capacità di interfacciarsi in modo nativo e sicuro con database PostgreSQL e API REST ha permesso di creare uno strumento di moderazione funzionale in poche ore, senza sottrarre tempo allo sviluppo dell’applicazione principale.

3.2.3 Version Control e Code Hosting

- **Git:** È il sistema di **controllo di versione distribuito** utilizzato per tracciare ogni modifica al codice sorgente del progetto. La sua adozione è stata fondamentale per gestire lo sviluppo in modo strutturato, permettendo la creazione di branch per nuove funzionalità, la gestione dei merge e la possibilità di tornare a versioni precedenti del codice.
- **GitHub:** È la piattaforma di **code hosting** che ospita il repository Git del progetto. Oltre al semplice hosting, è stata utilizzata per la sua integrazione con la pipeline di CI/CD di Vercel e, come descritto nel capitolo sulla distribuzione, per ospitare in modo sicuro le release dell’applicazione Android (APK).

3.2.4 Strumenti di Documentazione e Modellazione

- **PlantUML**: È lo strumento open-source utilizzato per creare tutti i diagrammi UML e architetturali. La sua caratteristica distintiva è l’approccio “diagrams as code”: invece di disegnare manualmente i diagrammi, questi vengono descritti attraverso un **linguaggio DSL (Domain-Specific Language)** testuale. Questa scelta ha permesso di creare diagrammi versionabili, facilmente modificabili e stilisticamente coerenti.

Progettazione e implementazione del sistema

4.1 Linee guida progettuali

La progettazione di **Swift Contest** è stata guidata da principi di ingegneria del software moderni, volti a garantire un sistema robusto, sicuro e manutenibile, anche in un contesto di sviluppo rapido. Le linee guida principali sono state:

- **Separazione delle responsabilità (SoC):** è stata applicata una netta separazione tra il livello di presentazione (UI), la logica di presentazione (ViewModel) e il livello di accesso ai dati (Model/Repository), fondamentale per la manutenibilità e scalabilità del codice.
- **Sicurezza come requisito primario:** è stato adottato un modello di accesso ai dati basato sul **Principio del Minimo Privilegio (Principle of Least Privilege)**, dove il client è considerato intrinsecamente non sicuro. L'accesso al database è negato per default e concesso selettivamente solo tramite un'API sicura.
- **Architettura Client-Server con backend BaaS:** è stata adottata una moderna architettura Client-Server. Il client (sviluppato in Flutter) gestisce la presentazione e la logica di UI, mentre il server è rappresentato da una piattaforma **Backend-as-a-Service (BaaS)** come Supabase. Questo approccio permette di

delegare la gestione dell'infrastruttura (database, autenticazione, storage) e di distribuire la logica di business in modo flessibile tra il client e le funzioni serverless (RPC ed Edge Functions).

- **Sviluppo cross-platform:** la scelta di un framework come Flutter ha permesso di massimizzare il riuso del codice tra le piattaforme target (Android e Web), ottimizzando i tempi di sviluppo.

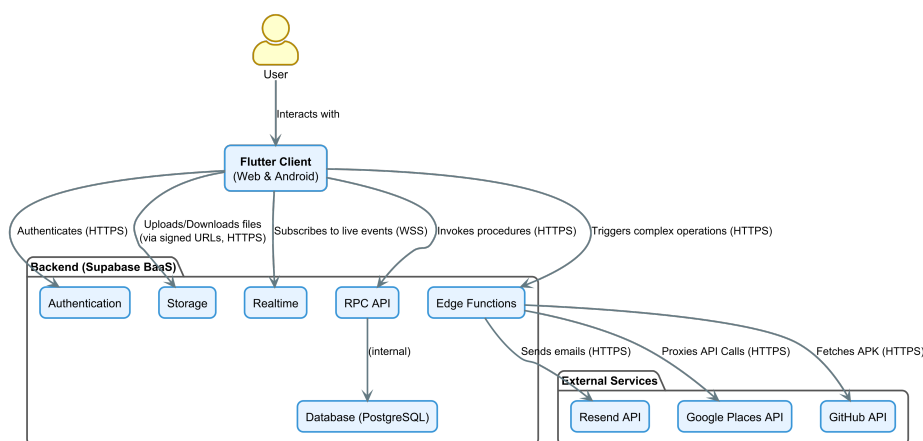


Figura 4.1: Architettura Client-Server del sistema Swift Contest con backend BaaS.

4.2 Architettura del frontend: il pattern MVVM con BLoC

Il frontend è stato sviluppato in Flutter seguendo i principi dell'architettura **MV-VM (Model-View-ViewModel)**, implementata attraverso il pattern **BLoC (Business Logic Component)**. Questa scelta permette di ottenere una netta separazione delle responsabilità, garantendo testabilità e manutenibilità. La struttura del codice sorgente in `lib/` riflette questa suddivisione:

- **View (/view):** il livello di presentazione, composto dai Widget di Flutter che costituiscono l'interfaccia utente. Questo livello si limita a mostrare i dati ricevuti dal suo ViewModel (il BLoC) e a notificare gli eventi di interazione dell'utente (es. la pressione di un pulsante).

- **ViewModel (/viewmodel, implementato come BLoC)**: il cuore della logica di presentazione. Ogni BLoC riceve eventi dalla View, esegue la logica di business (spesso invocando un metodo del Model/Repository), e produce uno stream di stati a cui la View reagisce per aggiornarsi.
- **Model (/model)**: il livello di accesso ai dati. È composto dai *Repository*, che fungono da unica interfaccia verso il backend, astruendo le chiamate a RPC ed Edge Functions. Include anche le *Entities* (le rappresentazioni dei dati del dominio) e i *Bundles* (oggetti aggregati per ridurre il numero di chiamate di rete).

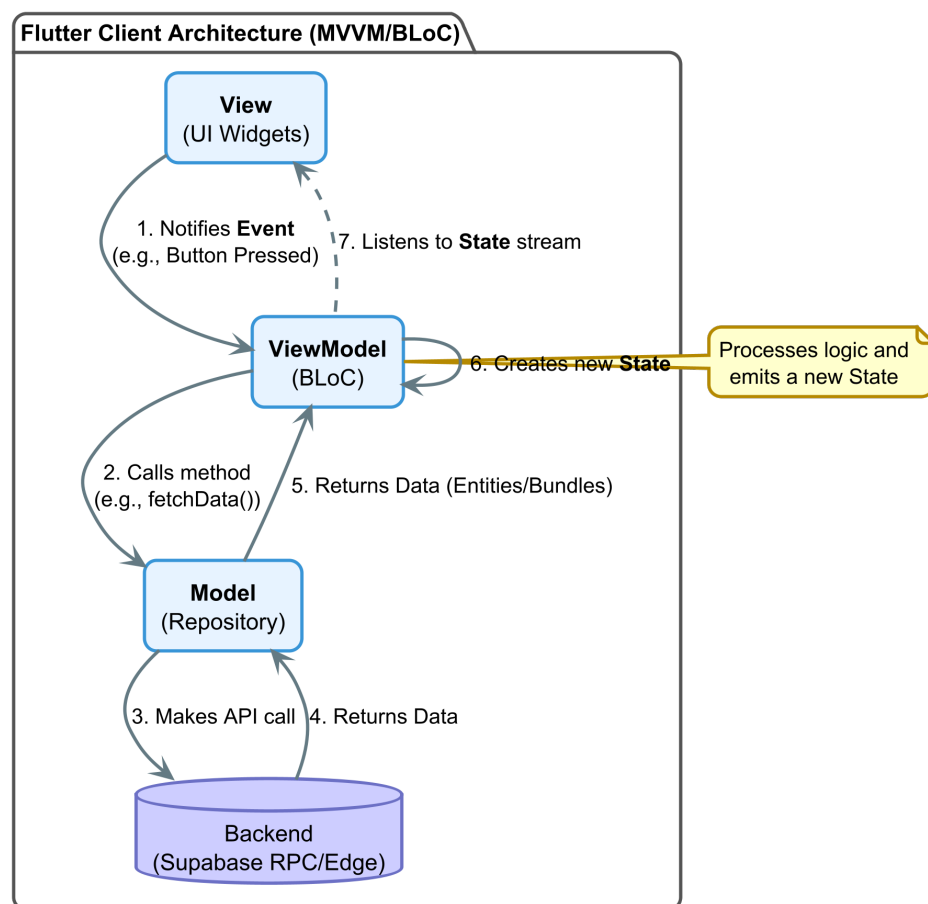


Figura 4.2: Architettura del Client MVVM con BLoC.

4.2.1 Implementazione del ViewModel: il pattern BLoC

Per la gestione dello stato e l'implementazione del ViewModel, è stato adottato il pattern BLoC, utilizzando il pacchetto `flutter_bloc`. La scelta di BLoC è motivata

dalla sua natura reattiva e basata su stream, che si integra perfettamente con il paradigma dichiarativo di Flutter. Questo pattern impone un flusso di dati unidirezionale e prevedibile:

1. La **View** notifica un **Evento** (es. `LoginButtonPressed`).
2. Il **BLoC** riceve l'evento, esegue la logica di business (es. chiama `authRepository.login` e, al termine, emette un nuovo **Stato** (es. `LoginSuccess` o `LoginFailure`).
3. La **View**, tramite un widget come `BlocBuilder`, si mette in ascolto dello stream di stati e si ridisegna per riflettere il nuovo stato.

Questo approccio garantisce che la logica di business sia completamente disaccoppiata dalla UI, rendendola facilmente testabile in isolamento. I BLoC sono stati organizzati in due categorie: **globali** (es. `AuthBloc`, `InboxBloc`) e **di pagina** (es. `JurorVotingProcedurePageBloc`).

Gestione centralizzata degli errori nel repository

Una delle responsabilità principali del livello *Model* (e in particolare dei *Repository*) è quella di agire come un robusto intermediario tra la logica di business e il backend. Questo include non solo il recupero dei dati, ma anche una **gestione centralizzata e sicura degli errori**.

Le chiamate a un backend come Supabase possono fallire in molti modi: problemi di rete (`SocketException`), errori di autenticazione (`AuthException`), errori del database (`PostgreSQLException`), ecc. Esporre queste eccezioni direttamente al livello del BLoC o della UI sarebbe problematico per due motivi:

1. Creerebbe un forte accoppiamento tra la logica di presentazione e le librerie specifiche del backend.
2. Potrebbe esporre all'utente finale messaggi di errore tecnici e potenzialmente sensibili.

Per risolvere questo problema, è stata creata una funzione di ordine superiore, `handleBackendCall`, che “avvolge” ogni chiamata al backend. Come mostrato nel listato seguente, questa funzione utilizza un blocco `try...catch` per intercettare

le eccezioni specifiche di Supabase e mapparle in classi di **Failure** personalizzate e sicure. Queste *Failure* sono oggetti semplici che rappresentano un fallimento in modo astratto (es. `NetworkFailure`, `ServerFailure`), senza contenere dettagli tecnici.

Il risultato di questa mappatura viene quindi incapsulato utilizzando il tipo **Either**, fornito dal pacchetto di programmazione funzionale `fpdart`. `Either` è un tipo di dato che può contenere solo uno di due valori possibili: un valore di “sinistra” (*Left*), che per convenzione rappresenta il fallimento, o un valore di “destra” (*Right*), che rappresenta il successo.

Il tipo di ritorno `Either<Failure, T>` formalizza quindi questo contratto: la funzione restituirà o un oggetto di tipo `Failure` (a sinistra) o il dato atteso di tipo `T` (a destra). Questo approccio elimina la necessità per il codice chiamante (il BLoC) di usare blocchi `try...catch`, permettendogli di gestire in modo pulito, dichiarativo e fortemente tipizzato sia il caso di successo che quello di fallimento, semplicemente ispezionando il contenuto di `Either`.

```

1  /// Avvolge una chiamata al backend, gestendo e mappando le eccezioni.
2  Future<Either<Failure, T>> handleBackendCall<T>(
3    Future<Either<Failure, T>> Function() function,
4  ) async {
5    try {
6      // Esegue la funzione che contiene la chiamata al backend.
7      return await function();
8    } on SocketException catch (e) {
9      // Errore di rete (es. no internet).
10     Logger.warning('Network error: $e');
11     return Either.left(const NetworkFailure());
12   } on AuthException catch (e) {
13     // Errore di autenticazione di Supabase.
14     Logger.warning('Auth error: $e');
15     return Either.left(authExceptionToFailure(e));
16   } on PostgreSQLException catch (e) {
17     // Errore del database PostgreSQL.
18     Logger.warning('Postgre error: $e');
19     return Either.left(postgreSQLExceptionToFailure(e));
20   } on FunctionException catch (e) {
21     // Errore durante l'invocazione di una Edge Function.
22     Logger.warning('Edge function error: $e');
23     return Either.left(edgeFunctionExceptionToFailure(e));
24   } on StorageException catch (e) {
25     // Errore nello Storage (es. upload fallito).

```

```
26   Logger.warning('Storage error: $e');
27   return Either.left(storageExceptionToFailure(e));
28 } catch (e) {
29   // Qualsiasi altro errore non previsto.
30   Logger.error('Unknown error: $e');
31   return Either.left(const Failure());
32 }
33 }
```

Listing 1: Funzione helper per la gestione sicura delle chiamate al backend.

Gestione dell'uguaglianza degli oggetti con Equatable

Per ottimizzare le performance e garantire un comportamento prevedibile nella gestione dello stato, tutte le entità, i bundle e gli stati dei BLoC estendono la classe fornita dal pacchetto `equatable`.

Questa scelta implementativa è cruciale per due motivi principali.

Il primo riguarda il corretto funzionamento del pattern BLoC. Di default, Dart confronta gli oggetti per riferimento, non per valore. Ciò significa che due istanze di un oggetto, anche se contengono dati identici, vengono considerate diverse. Senza `Equatable`, un BLoC potrebbe emettere un nuovo stato identico al precedente, causando una ricostruzione non necessaria e dispendiosa dell'interfaccia utente. Utilizzando `Equatable`, invece, il confronto avviene basandosi sul valore degli attributi dell'oggetto. Questo permette ai widget di Flutter, come `BlocBuilder`, di determinare in modo efficiente se uno stato è *realmente* cambiato prima di procedere con la ricostruzione, prevenendo refresh superflui della UI e migliorando significativamente la reattività dell'applicazione.

Il secondo motivo, altrettanto importante, riguarda la **robustezza e la chiarezza della logica di business**. Spesso, durante lo sviluppo, sorge la necessità di confrontare due oggetti per verificare se rappresentano la stessa entità concettuale, anche se sono istanze diverse in memoria. Estendendo `Equatable`, si ottiene un'implementazione standard e priva di errori dell'uguaglianza basata sul valore, semplificando la logica di confronto all'interno dei BLoC e dei Repository e riducendo il rischio di bug sottili legati al confronto per riferimento.

4.2.2 Navigazione con AutoRoute

La gestione della navigazione è stata affidata al pacchetto `auto_route`, che genera un router fortemente tipizzato, eliminando l'uso di stringhe per le rotte e riducendo il rischio di errori a runtime. Un elemento chiave è l'`AuthGuard`, una classe che intercetta le richieste di navigazione e verifica lo stato di autenticazione dell'utente tramite l'`AuthBloc`, reindirizzandolo alla pagina di login se tenta di accedere a una rotta protetta.

[language=Dart, caption=Estratto del sistema di routing protetto con `AuthGuard`, label=lst:auth_guard] // — — — 1. Definizione della rotta protetta (in `app_router.dart`) — — —

```
@AutoRouterConfig() class AppRouter extends RootStackRouter // ... @override
List<AutoRoute> get routes => [
  // ... altre rotte pubbliche o di autenticazione
  // Esempio di rotta protetta: solo gli utenti autenticati // possono accedere alla
  dashboard dell'organizzatore. AutoRoute( path: '/organizer-home', page: OrganizerHomeRoute.page, guards: [AuthGuard(authBloc: authBloc)], // Applica la guardia
),
  // ... altre rotte protette ];
```

// — 2. Implementazione della guardia (in `auth_guard.dart`) — — —

```
class AuthGuard extends AutoRouteGuard final AuthBloc authBloc;
AuthGuard(required this.authBloc);
@override void onNavigation(NavigationResolver resolver, StackRouter router)
// Controlla lo stato di autenticazione tramite l'AuthBloc final isAuthenticated =
authBloc.state.authStatus.isAuthenticated;
  // Elenco completo delle rotte pubbliche (di autenticazione) const authPaths
= [ '/sign-in', '/sign-in-verify', '/sign-up', '/sign-up-verify' ]; final targetPath =
resolver.route.path.split('/:').first;
  if (isAuthenticated) // Se l'utente è autenticato e cerca di andare a una pagina di
login/registrazione, // lo reindirizzo alla sua home page. if (authPaths.contains(targetPath))
resolver.redirect(const RootRoute()); else // Altrimenti, può procedere verso la
rotta protetta. resolver.next(true); else // Se l'utente NON è autenticato... if (!authPaths.contains(targetPath)) // ...e sta cercando di accedere a una rotta protet-
```



```
ta, // lo reindirizzo alla pagina di login. resolver.redirect(SignInRoute()); else
// ...e sta già andando a una pagina di login/registrazione, lo lascio procedere.
resolver.next(true);
```

4.2.3 Strategia di navigazione e passaggio dei dati

Una scelta architetturale fondamentale, dettata dalla necessità di una perfetta integrazione con il web, riguarda la modalità di passaggio dei dati tra le pagine. A differenza di un'applicazione puramente mobile, dove è comune passare interi oggetti in memoria durante la navigazione, in un'applicazione web questo approccio è insostenibile: l'utente può ricaricare la pagina o accedere a un URL diretto, perdendo qualsiasi stato tenuto in memoria.

Per questo motivo, il sistema di navigazione di Swift Contest adotta una strategia rigorosa: **nelle rotte vengono passati solo identificativi (ID) primitivi** (es. `contestId`, `juryId`), mai oggetti complessi. Quando una pagina viene caricata, il BLoC associato riceve l'ID dai parametri della rotta e, come prima azione, invoca il Repository per caricare dal backend tutti i dati necessari.

Questa scelta, sebbene aumenti il numero di chiamate al database, garantisce vantaggi in termini di robustezza e coerenza con il paradigma web:

- Ogni pagina è **ricaricabile** in modo indipendente.
- Ogni pagina ha un **URL univoco e condivisibile** (deep linking).
- Il comportamento dell'applicazione è prevedibile e consistente tra la navigazione interna e l'accesso diretto a un URL.

Validazione dell'input utente e gestione dei form

Per garantire l'integrità dei dati e fornire un feedback immediato all'utente, è stata implementata una robusta strategia di validazione dell'input direttamente a livello client, prima che i dati vengano inviati al BLoC e, successivamente, al backend.

Invece di definire la logica di validazione all'interno dei singoli widget, **tutte le funzioni di validazione sono state centralizzate** in un unico file, `lib/utils/validators/validators.dart`. Questo approccio, basato su funzioni pure, offre due vantaggi significativi:

- **Riutilizzabilità:** La stessa funzione, ad esempio `emailValidator`, può essere riutilizzata in più punti dell'applicazione (es. pagina di login, registrazione, modifica profilo) senza duplicazione di codice.
- **Manutenibilità:** Se una regola di validazione deve essere modificata (es. la complessità minima di una password), la modifica viene effettuata in un unico punto, garantendo coerenza in tutta l'applicazione.

Il listato seguente mostra un esempio di un *validator* e il suo utilizzo all'interno di un widget `TextFormField`.

[language=Dart, caption=Esempio di validatore centralizzato e suo utilizzo in un widget., label=lst:validators] // 1. Definizione della funzione di validazione pura // in lib/utils/validators/validators.dart

```
String? passwordValidator(String? value) if (value == null || value.isEmpty)
return 'Required'; if (value.length < 8) return 'Must contain at least 8 characters';
if (!RegExp(r'[A-Z]').hasMatch(value)) return 'Must contain at least one uppercase
letter'; if (!RegExp(r'[0-9]').hasMatch(value)) return 'Must contain at least one
number'; if (!RegExp(r'[!@* return 'Must contain at least one special character';
return null; // La validazione ha successo
```

// 2. Utilizzo del validatore in un widget della UI

```
TextFormField( // ... altre proprietà come controller, decoration, etc.
```

```
// La funzione di validazione viene passata come riferimento. validator: pas-
swordValidator,
```

```
// La validazione viene attivata automaticamente al momento // del submit del
form o in base all'autovalidateMode. autovalidateMode: AutovalidateMode.onUserInteraction,
);
```

4.2.4 Design system e theming

Per garantire un'interfaccia utente coerente e facilmente manutenibile, è stato definito un design system centralizzato basato sul **Material Design 3**. La scelta di `MaterialTheme` come base è stata strategica, in quanto è lo standard de facto per le applicazioni Android e garantisce un'esperienza utente nativa e riconoscibile.

Il cuore del sistema di theming di Material Design è il **ColorScheme**, un oggetto che definisce una palette di colori predefinita e semanticamente organizzata. Invece di usare colori specifici i widget fanno riferimento a ruoli di colore come `primary` (per elementi principali), `secondary` (per elementi di supporto), `surface` (per gli sfondi), `onPrimary` (per il testo da mostrare sopra un colore primario), ecc. Questo approccio permette di cambiare l'intera palette di colori dell'applicazione (ad esempio, passando dal tema chiaro a quello scuro) modificando unicamente l'oggetto `ColorScheme`, senza dover alterare il codice dei singoli widget.

Tuttavia, il `ColorScheme` standard non è sufficiente per coprire tutte le esigenze di un'applicazione complessa, che richiede colori semantici personalizzati (es. per stati di successo, errore, warning).

Per risolvere questo problema in modo pulito e scalabile, invece di definire colori statici sparsi nel codice, si è fatto ricorso a una funzionalità del linguaggio Dart: le **estensioni**. Un'estensione (*extension*) permette di **aggiungere nuove funzionalità a una classe esistente**, anche se non si ha accesso al suo codice sorgente. In questo caso, è stata creata un'estensione sulla classe `ColorScheme` di Flutter per "arricchirla" con i colori semantici personalizzati richiesti dall'applicazione.

Come mostrato nel listato seguente, l'estensione `ColorSchemeX` aggiunge nuovi *getter* (es. `success`, `warning`) alla classe `ColorScheme`. Questi nuovi colori diventano così parte integrante del tema e possono essere richiamati in qualsiasi punto dell'applicazione in modo sicuro e coerente tramite `Theme.of(context).colorScheme.success` ereditando automaticamente i cambiamenti tra tema chiaro e scuro.

[language=Dart, caption=Definizione dell'estensione `ColorSchemeX` per aggiungere colori personalizzati al tema., label=lst:colorscheme_extension] // inlib/Utils/themes/color_s

```
extension ColorSchemeX on ColorScheme // Colore per indicare successo (es.
notifiche positive) Color get success => const Color(0xFF28a745); Color get onSuccess
=> const Color(0xFFFFFFFF);
```

```
// Colore per avvisi Color get warning => const Color(0xFFfc107); Color get
onWarning => const Color(0xFF212529);
```

```
// Altri colori personalizzati...
```

Gestione del feedback di caricamento: l'OverlayLoader

Per fornire un feedback visivo all'utente durante le operazioni asincrone, è stato implementato un sistema di caricamento personalizzato, `OverlayLoader`. Questo sistema utilizza l'Overlay di Flutter per mostrare un indicatore di progresso con uno sfondo semitrasparente e offuscato, permettendo all'utente di mantenere il contesto visivo della schermata sottostante.

L'implementazione si basa su due scelte di design chiave per garantire robustezza e facilità d'uso:

- **Singleton Pattern:** La classe `OverlayLoader` è implementata come un singleton per garantire che esista una sola istanza del gestore del loader, prevenendo la visualizzazione di più overlay contemporaneamente.
- **Dart Extensions:** Per semplificare l'invocazione del loader, è stata creata un'estensione sulla classe `BuildContext`, che permette di usare una sintassi pulita e intuitiva come `context.showLoader()` e `context.hideLoader()`.

Gestione del ciclo di vita e prevenzione dei "leak" Un aspetto cruciale per la robustezza di questo sistema è la gestione del suo ciclo di vita. Per evitare che l'overlay del loader rimanga "bloccato" sullo schermo se una pagina viene chiusa (*disposed*) durante un'operazione di caricamento (ad esempio, a causa di un errore o della navigazione dell'utente), è stata adottata una pratica difensiva: **il metodo `context.hideLoader()` viene sistematicamente invocato all'interno del metodo `dispose` di ogni pagina** che utilizza il loader. Questo garantisce che, indipendentemente da come o perché la pagina viene distrutta, qualsiasi overlay di caricamento attivo venga sempre rimosso, prevenendo "leak" visuali e garantendo la stabilità dell'interfaccia utente.

```
[language=Dart, caption=Implementazione del loader non-intrusivo tramite Overlay e Dart Extensions., label=lst:overlay_loader] // 1. Classe Singleton per la gestione dell'overlay class
// Singleton pattern per garantire un'unica istanza OverlayLoader.(); static final OverlayLoad
OverlayLoader.(); factory OverlayLoader() => _instance;
void show(BuildContext context) if (_entry! = null) return; // Evita di mostrare più loader
```

```
entry = OverlayEntry(builder : () => const Positioned.fill( // ObscuredLoader widget custom  
ObscuredLoader(), ), );  
// Inserisce l'entry nell'overlay radice per essere sempre in cima Overlay.of(context,  
rootOverlay: true).insert(entry!);  
void hide() entry?.remove(); entry = null;  
// 2. Estensione su BuildContext per un'API pulita e intuitiva extension Overlay-  
LoaderX on BuildContext void showLoader() => OverlayLoader().show(this); void  
hideLoader() => OverlayLoader().hide();
```

4.3 Progettazione del database

Il modello concettuale del dominio, descritto nel Capitolo 2, è stato tradotto in un modello logico e fisico implementato su un database **PostgreSQL**, gestito tramite la piattaforma Supabase. La progettazione si è concentrata su tre pilastri: integrità dei dati, sicurezza e performance.

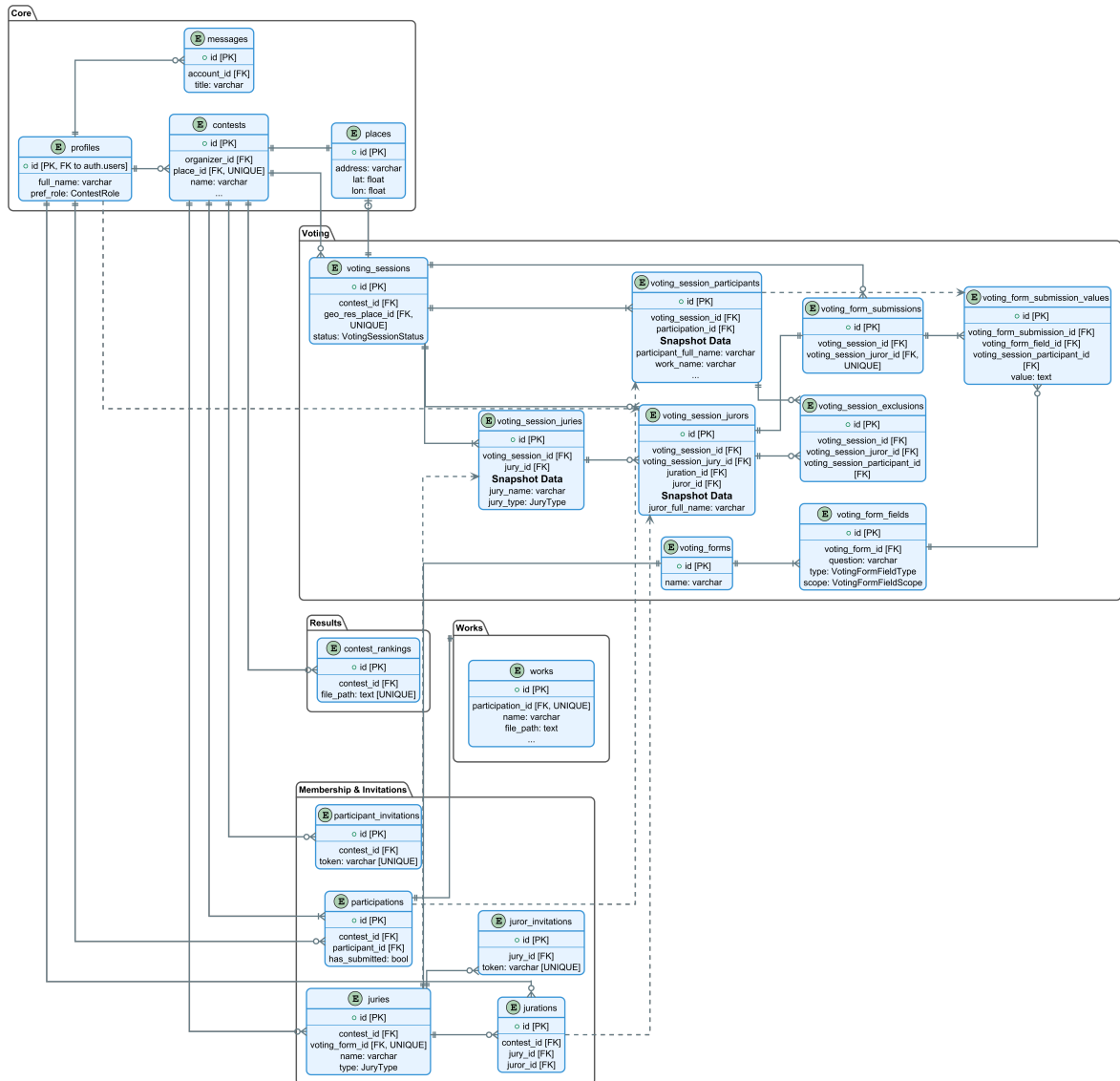


Figura 4.3: Diagramma Entità-Relazioni (ERD) dello schema del database.

4.3.1 Schema fisico e tabelle principali

Lo schema del database è composto da una serie di tabelle che implementano le entità del dominio. Le tabelle cardine includono:

- **profiles:** Questa tabella estende la tabella `auth.users` gestita internamente da Supabase. Poiché lo schema `auth` è protetto e non modificabile, per aggiungere attributi personalizzati (es. `full_name`, `pref_role`) è stata creata una tabella parallela nello schema `public`. La colonna `id` di `profiles` funge sia da chiave primaria che da chiave esterna verso `auth.users(id)`, stabilendo

una relazione uno-a-uno. La creazione del profilo è automatizzata tramite un **trigger** a livello di database, che inserisce un nuovo record in `profiles` ogni volta che un utente completa la registrazione.

- `contests`: memorizza le informazioni principali di un concorso.
- `participations` e `jurations`: implementano le classi di associazione che legano utenti, contest e giurie.
- `works`: contiene i metadati delle opere sottomesse dai partecipanti.
- `voting_forms` e `voting_form_fields`: definiscono la struttura dei moduli di voto.
- `voting_sessions`: rappresenta una sessione di voto.
- `voting_session_participants`, `voting_session_jurors`: rappresentano i membri coinvolti in una specifica sessione di voto, oltre a contenere gli snapshot dei dati per quella sessione.
- `voting_form_submissions` e `voting_form_submission_values`: memorizzano i voti inviati dai giurati.

4.3.2 Scelte di design chiave

Oltre alla semplice mappatura delle entità, sono state prese decisioni di design cruciali per garantire la robustezza del sistema.

Integrità referenziale e strategia di cancellazione

Per garantire la consistenza del database, è stato fatto un uso estensivo dei vincoli di chiave esterna. Una decisione di design fondamentale è stata quella di adottare una strategia di **cancellazione fisica (hard delete)** piuttosto che una di **cancellazione logica (soft delete)**, principalmente per ottimizzare l'uso delle risorse (spazio su database e storage) e per semplificare la logica delle query di lettura.

Data questa scelta, la gestione della cancellazione dei dati correlati è diventata un aspetto critico, risolto con un approccio a più livelli.

Cancellazione a cascata tramite ON DELETE CASCADE Per le relazioni di composizione forte e dirette, dove l'entità "figlia" non ha dipendenze esterne come file nello Storage, è stata utilizzata la politica **ON DELETE CASCADE**. L'esempio più calzante è la relazione tra `voting_forms` e `voting_form_fields`. Poiché i campi di un form non hanno senso di esistere senza il form stesso, l'eliminazione di un record in `voting_forms` provoca la cancellazione automatica e a cascata di tutti i record correlati in `voting_form_fields`. Questo approccio delega interamente la logica di pulizia al database, garantendo massima efficienza e consistenza dei dati con il minimo sforzo implementativo.

Cancellazione tramite trigger per relazioni uno-a-uno In alcune relazioni uno-a-uno, la chiave esterna è stata posizionata nella tabella concettualmente "principale" per motivi di design. L'esempio più significativo è la relazione tra `contests` e `places`: è la tabella `contests` a contenere il riferimento `place_id` con un vincolo `UNIQUE`.

Questa configurazione, sebbene sia logicamente corretta, impedisce di usare la politica **ON DELETE CASCADE** per eliminare un `place` quando il `contest` associato viene cancellato. Per risolvere questo problema e automatizzare la pulizia dei dati, è stato implementato un **trigger di database**. Questo trigger si attiva *dopo* (`AFTER`) l'eliminazione di un record dalla tabella `contests` ed esegue esplicitamente la cancellazione del record corrispondente nella tabella `places`, utilizzando l'ID del luogo del contest appena eliminato. In questo modo, si garantisce l'integrità referenziale e si evita di lasciare record `places` orfani, delegando anche in questo caso la logica di pulizia al database.

Cancellazione di dati esterni tramite Edge Functions La cancellazione a cascata del database non può gestire dati esterni, come i file memorizzati in Supabase Storage. Per questo motivo, le operazioni di cancellazione più complesse, come l'eliminazione di un utente o di un contest, sono gestite da **Edge Functions** dedicate (es. `user-delete-account`, `admin-delete-contest`). Queste funzioni agiscono da orchestratori: dopo aver eliminato i record dal database (che a loro volta attivano le cancellazioni a cascata e i trigger), invocano l'API di Supabase Storage per

eliminare tutti i file associati (es. immagini del contest, file dei *Work*), garantendo una pulizia completa e prevenendo la comparsa di file “orfani”. Questa logica non poteva essere implementata in una funzione RPC, poiché queste non hanno accesso all’API dello Storage.

Snapshot dei dati tramite denormalizzazione controllata

Per garantire l’immutabilità e l’auditabilità del processo di voto, è stato implementato un meccanismo di **snapshot dei dati**, una delle scelte architetturali più importanti del backend.

Quando un organizzatore avvia una *VotingSession* tramite la funzione RPC `organizer_start_session`, il sistema non si limita a collegare le entità esistenti, ma esegue una forma di **denormalizzazione controllata** (si aggiungono volutamente informazioni ridondanti nel database). Per ogni partecipante e giurato coinvolto, vengono create nuove righe in tabelle di sessione dedicate (es. `voting_session_participants`, `voting_session_jurors`).

Queste nuove righe contengono non solo le chiavi esterne ai record originali, ma anche una copia statica degli attributi più importanti al momento dell’avvio. Come mostrato nel listato seguente, la tabella `voting_session_participants` include, oltre a `participation_id`, anche i campi `participant_full_name`, `work_name`, ecc.

Una scelta di design cruciale è l’uso della clausola **ON DELETE SET NULL** sulla chiave esterna `participation_id`. Questo significa che se il partecipante originale dovesse essere eliminato dal sistema, il record dello snapshot *non* verrebbe cancellato, ma manterrebbe solo il riferimento a `NULL`. Ciò dimostra che la vera “fonte di verità” per la sessione di voto sono gli attributi di snapshot, garantendo che i voti espressi rimangano validi e auditabili anche se le entità originali cessano di esistere.

Lo stesso principio si applica a `voting_session_jurors`, dove `juror_id` è fondamentale per tracciare se un giurato ha già votato, ma può essere impostato a `NULL` in caso di cancellazione dell’account senza invalidare il voto già espresso.

Questa tecnica garantisce:

1. **Integrità della sessione:** se un utente cambia nome, il nome “congelato” nello snapshot rimane invariato.
2. **Auditabilità e resilienza:** i voti rimangono validi e associati ai dati di snapshot anche se i record originali vengono eliminati.
3. **Performance ottimizzate:** i dati di snapshot vengono letti direttamente, evitando query con join.

[language=SQL, caption=Definizione delle tabelle di sessione con attributi di snapshot e gestione della cancellazione., label=lst:snapshot_aattributes] – — Definizione della tabella ‘voti’

```
CREATE TABLE public.voting_session_participants(iduuuidPRIMARYKEYDEFAULTExtensions.uu
  – Riferimento opzionale al record di partecipazione originale. – ON DELETE SET
NULL: se il partecipante viene rimosso dal contest, – lo snapshot sopravvive, garan-
tendo l’integrità dei voti. participation_iduuuidREFERENCESpublic.participations(id)ONDELE
  order_indexintNOTNULL,
  – — Attributi di Snapshot — – Dati "congelati" al momento dell’avvio della sessio-
ne. participant_full_namevarchar(70)NOTNULL, work_namevarchar(50)NOTNULL, work_descript
  UNIQUE(voting_session_id, participation_id), UNIQUE(voting_session_id, order_index));
  – Definizione della tabella ‘voting_session_jurors’
CREATE TABLE public.voting_session_jurors(iduuuidPRIMARYKEYDEFAULTExtensions.uu
  – Riferimento opzionale al giurato. – ON DELETE SET NULL: se l’account del giu-
rato viene eliminato, – il suo voto rimane registrato ma anonimizzato. juror_iduuuidREFERENCES
  has_submittedboolNOTNULLDEFAULTfalse,
  – — Attributo di Snapshot — juror_full_namevarchar(70)NOTNULL,
  UNIQUE(voting_session_jury_id, juror_id));
```

4.3.3 Implicazioni del design del form di voto sull’aggregazione dei risultati

Una scelta di design fondamentale, visibile nello schema del database, è che **ogni giuria è associata a un unico e specifico form di voto**. Questa flessibilità permette all’organizzatore di creare criteri di valutazione diversi per giurie diverse (es. una

giuria tecnica con campi slider complessi e una giuria popolare con un singolo campo di preferenza).

Tuttavia, questa decisione architetturale ha un’implicazione diretta e importante su come i risultati possono essere processati e visualizzati, influenzando sia il backend che il frontend: **il sistema non può aggregare automaticamente i punteggi tra giurie diverse**, poiché i loro form di voto (e quindi i loro criteri e punteggi) non sono direttamente comparabili.

Di conseguenza, tutte le operazioni di analisi dei risultati, sia a livello di logica di backend (RPC) che di interfaccia utente (frontend), sono state progettate per operare a livello della singola giuria:

- La **generazione della classifica** avviene per ogni singola giuria.
- L'**esportazione dei risultati** in formato Excel è granulare e produce un file per ciascuna giuria.

Se l’organizzatore desidera creare una classifica “globale” che combini i risultati di più giurie, deve farlo esternamente, utilizzando i dati esportati. Questa è una limitazione consapevole del progetto, un compromesso accettato per garantire la massima flessibilità nella configurazione dei criteri di valutazione.

Per mantenere questa flessibilità anche nella fase finale, il sistema non impone una classifica globale, ma permette all’organizzatore di **pubblicare qualsiasi classifica in formato PDF**. Questa scelta di design gli conferisce la **massima libertà**: può decidere di pubblicare la classifica di una singola giuria, oppure può elaborare esternamente una classifica globale personalizzata e caricarla come file PDF. In entrambi i casi, una volta pubblicata, la classifica diventa disponibile per il download a tutti i partecipanti e giurati del contest all’interno dell’applicazione.

4.4 Architettura del backend

Il backend è costruito interamente su Supabase e la sua architettura è definita da un’API sicura composta da un insieme di Funzioni RPC ed Edge Functions.

4.4.1 Funzioni RPC (Remote Procedure Call)

Le funzioni RPC, scritte in PL/pgSQL, sono il principale punto di ingresso per le operazioni sul database. La scelta di utilizzare le RPC come strato di accesso ai dati primario, invece di eseguire query dirette dal client, è stata dettata da tre motivi fondamentali:

1. **Sicurezza e Controllo:** Sono tutte dichiarate con `SECURITY DEFINER`, il che significa che vengono eseguite con i privilegi del loro creatore (un ruolo amministrativo) e non con quelli dell'utente chiamante. Al loro interno, contengono logica di validazione e controllo dei permessi, tipicamente verificando l'identità dell'utente tramite `auth.uid()`. Questa scelta è il fulcro dell'approccio di **accesso mediato da API**: nessun utente può effettuare operazioni dirette sul database, ma deve passare attraverso questo strato di controllo.
2. **Supporto per le Transazioni:** Una limitazione chiave della libreria client `supabase-flutter` è la **mancanza di supporto per le transazioni multi-statement**. Operazioni complesse che richiedono l'inserimento o l'aggiornamento di record su più tabelle in modo atomico (es. la creazione di un contest e del suo luogo associato) non sarebbero sicure se eseguite tramite chiamate multiple dal client. Implementando questa logica all'interno di una singola funzione RPC, si sfrutta la capacità nativa di PostgreSQL di eseguire tutte le operazioni in un'unica transazione, garantendo che l'operazione abbia successo o fallisca nella sua interezza, senza lasciare il database in uno stato inconsistente.
3. **Performance e Astrazione:** Le RPC permettono di incapsulare logica complessa e query con join multipli lato server, riducendo la quantità di dati trasferiti e il numero di round-trip tra client e server.

Nome Funzione	Scopo Principale	Controllo di Sicurezza Esempio
<code>auth_get_account_bundle</code>	Ottiene i dati completi dell'utente autenticato.	<code>auth.uid() = id</code>

Nome Funzione	Scopo Principale	Controllo di Sicurezza Esempio
<code>organizer_create_contest</code>	Crea un nuovo contest e il luogo associato.	Verifica che l'utente sia autenticato.
<code>organizer_start_voting_session</code>	Avvia una sessione di voto, creando lo snapshot dei dati.	L'utente deve essere l'organizzatore del contest.
<code>participant_join_contest</code>	Associa un partecipante a un contest tramite un token di invito.	Il token deve essere valido e non utilizzato.
<code>juror_submit_votes</code>	Salva i voti di un giurato per una sessione.	La sessione deve essere 'live' e il giurato non deve aver già votato.
<code>juror_access_voting_as_simple_juror</code>	Fornisce l'accesso a un giurato ospite tramite il token della giuria.	Il token della giuria deve essere valido.

Tabella 4.1: Elenco esemplificativo delle Funzioni RPC del sistema.

[language=SQL, caption=Estratto dalla funzione `juror_submit_votes` che mostra la validazione server-side dei valori degli slider., label=lst:juror_ssubmit_votes_vvalidation] –

–STEP2.5 : VALIDATEVOTEVALUES – –Controllainmodoefficientesequalchevaloreditipo'slider' –

–al di fuoridel suor angemin / maxdefinitoononun numerovalido.IF EXISTS(– – UtilizzaunaCom –

–in unatabellatemporaneadirigheecolonne.WITHunpacked_votesAS(SELECT(value – >>' –

voting_form_field_id') :: uuidasfield_id, (value – >>' value') :: textasvalueFROMjsonb_ar_ray_elemen –

–Selezionalerighecheviolanoleregoledivalidazione.SELECT1FROMunpacked_votesuv –

–Unisceivotispacchettaticonlatabelladeicampidel form – –perottenereimetadatidiognicampo(tip –

uv.field_idWHERE – –Applicailcontrollosoloaicampiditipo'slider'.vff.type =' slider' AND(– –

Condizione1 : Ilvalorenonunastringadiunnumerointero.NOT(uv.value '-'?+') – Con-

dizione 2: Oppure, se è un intero, è fuori dai limiti. OR $uv.value::integer < vff.slider_{min_value} OR uv.value::integer > vff.slider_{max_value}$) THEN — Se anche solo una riga viola le condizioni, solleva un'eccezione — e interrompe la transazione, impedendone l'inserimento dei dati. RAISE EXCEPTION 'Invalid vote value. A slider value is outside its allowed range or is not a valid number.'; ENDIF;

– Se la validazione passa, la funzione procede con l'inserimento dei voti...

4.4.2 Edge Functions

Le Edge Functions, scritte in TypeScript ed eseguite su Deno, gestiscono operazioni complesse, transazioni multi-step e integrazioni con servizi esterni che non sarebbero possibili o sicure da eseguire in una semplice RPC.

Nome Funzione	Scopo	Autorizzazione / Dipendenze
organizer-invite-jury	Invia un'email di invito a un giurato.	JWT + Ownership del contest; Resend API .
participant-submit-work	Gestisce il caricamento del file .zip del Work e delle immagini di anteprima.	JWT + Appartenenza al contest.
user-delete-account	Elimina un utente e tutti i suoi dati e file associati.	JWT (self-service).
admin-delete-account	Versione amministrativa dell'eliminazione account.	X-API-Key (per Re-tool).
google-places-search	Funge da proxy sicuro per l'API di Google Places.	JWT; Google Places API .
get-latest-apk	Proxy sicuro per il download dell'APK da un repository GitHub privato.	JWT; GitHub API .

Tabella 4.2: Elenco esemplificativo delle Edge Functions del sistema.

```

[language=TypeScript, caption=Estratto dalla Edge Function organizer-invite-juror
che mostra l'invocazione dell'API di Resend per l'invio di email., label=lst:resend_invocation] // In
resend';constresend = newResend(Deno.env.get('RESEND_API_KEY'));

// ... (all'interno della funzione Deno.serve, dopo aver validato l'input // e aver
creato il record di invito nel database)

try // ... // Passaggio 5: Inserisce il nuovo invito nel DB e ottiene il token
generato. const data: newInvitation, error: insertError = await supabaseAdmin
.from('juror_invitations').insert(contest_id, jury_id, email).select().single();

if (insertError) throw insertError;

// Passaggio 6: Recupera i nomi del contest e della giuria per personalizzare l'e-
mail. const data: contest = await supabaseAdmin.from('contests').select('name').eq('id',
contest_id).single();constdata : jury = awaitsupabaseAdmin.from('juries').select('name').eq('id',
// Passaggio 7: Invia l'email di invito utilizzando il client di Resend. // L'opera-
zione è asincrona. await resend.emails.send( from: "Swift Contest <noreply@swiftcontest.com>",
to: [email], subject: 'Invitation to join the jury for "contest.name"', // Ilcorpodell'emailuntemplate
' < div >< h1 > InvitationtoSwiftContest < /h1 >< p > Youhavereceivedaninvitationtojointh
strong >jury.name</strong>" jury for the contest "<strong>contest.name < /strong >
". < /p >< p > Usethistokenintheapptoaccept :< br >< strong >newInvitation.token</strong>
</p> </div> ', );

// Passaggio 8: Se l'invio ha successo, restituisce l'invito creato al client. return
new Response(JSON.stringify(newInvitation), headers: ...corsHeaders, "Content-
Type": "application/json" , status: 201, // Created );

catch (error) // Se l'invio dell'email o qualsiasi operazione precedente falli-
sce, // l'errore viene catturato e registrato, e viene restituita una risposta di er-
rore. console.error("Error sending invitation:", error.message); return new Respon-
se(JSON.stringify( error: error.message ), headers: ...corsHeaders, "Content-Type":
"application/json" , status: 500, );

```

4.5 Architettura di Sicurezza: Principio del Minimo Privilegio e Accesso Mediato

La sicurezza è un requisito trasversale che ha guidato l'intera progettazione del backend. È stato adottato un rigoroso modello basato sul **Principio del Minimo Privilegio (PoLP)**, il cui assunto fondamentale è negare l'accesso per default e concederlo solo in modo esplicito e controllato.

4.5.1 Strategia di Implementazione: Centralizzazione della Logica di Autorizzazione

Per implementare il Principio del Minimo Privilegio, si sarebbero potute seguire due strategie principali:

1. Definire policy di **Row Level Security (RLS)** granulari per ogni tabella e per ogni operazione (SELECT, INSERT, UPDATE, DELETE).
2. Centralizzare tutti i controlli di autorizzazione all'interno di un'API di accesso ai dati (RPC e Edge Functions).

Sebbene un approccio ibrido con un doppio livello di controllo (sia RLS che controlli nell'API) rappresenti la massima sicurezza teorica, per questo progetto è stata adottata la **seconda strategia**. La logica di autorizzazione è stata interamente centralizzata all'interno delle Funzioni RPC e delle Edge Functions. Questa scelta è stata motivata dalla necessità di ottimizzare i tempi di sviluppo e la manutenibilità: gestire una logica di autorizzazione complessa in un unico punto (il codice della funzione) è risultato più rapido e meno prone a errori rispetto a dover mantenere sincronizzate decine di policy RLS scritte in SQL.

4.5.2 Il Ruolo della RLS e dell'API come "Gatekeeper"

In questo modello, la RLS gioca comunque un ruolo fondamentale, ma più semplice: **agisce come un "firewall" di default**. Su tutte le tabelle è stata abilitata la RLS con una policy di `DENY` implicita. Questo garantisce che la chiave API usata dal client

Flutter (`anon_key`) non abbia, di per sé, alcun permesso diretto di leggere o scrivere dati.

L'accesso ai dati è quindi interamente mediato da un'API sicura:

- **Funzioni RPC SECURITY DEFINER:** per la maggior parte delle operazioni sul database.
- **Edge Functions:** per operazioni complesse e interazioni con servizi esterni.

Queste funzioni, agendo da “gatekeeper”, contengono al loro interno tutta la logica di autorizzazione (es. “l'utente è l'organizzatore di questo contest?”), verificando l'identità del chiamante tramite `auth.uid()` prima di eseguire qualsiasi operazione.

4.5.3 L'Eccezione Necessaria: le Policy RLS per il Servizio Realtime

Esiste un'eccezione mirata a questa strategia. Il servizio **Supabase Realtime** basa il suo meccanismo di autorizzazione proprio sulle policy RLS della tabella. Per permettere a un client di “ascoltare” le modifiche, è stato necessario definire policy di `SELECT` granulari, ma esclusivamente per le due tabelle esposte al servizio Realtime: `messages` e `voting_sessions`. Questo è stato un adattamento necessario per integrare una funzionalità chiave, mantenendo la massima sicurezza su tutte le altre operazioni e tabelle.

Il seguente listato mostra la policy implementata per la tabella `voting_sessions`.

```
[language=SQL, caption=Policy RLS di tipo SELECT per la tabella voting_sessions,
necessaria per il servizio Realtime., label=lst:rls_realtime_policy] -- Abilita l'accesso in lettura alla
-- solo per gli utenti autorizzati a ricevere aggiornamenti in tempo reale.CREATE POLICY "Allow read
Condizione1 : L'utente è l'organizzatore del contest a cui la sessione appartiene.(EXISTS(SELECT 1 FROM
voting_sessions contest_id AND c.organizer_id = auth.uid()))OR -- Condizione2 : L'utente è un giuratore
voting_sessions.id AND v.sj.juror_id = auth.uid())));
```

4.5.4 Misure di Sicurezza Aggiuntive

Oltre al modello ibrido RLS/RPC, la sicurezza del sistema è rafforzata da altre misure:

- **Protezione delle API Esterne:** le chiavi segrete per servizi come Resend e Google Places sono memorizzate in Supabase e accessibili solo dalle Edge Functions.
- **Protezione dei file:** nessun file nello Storage è pubblico. L'accesso avviene solo tramite *signed URL* temporanei generati lato server.

Gestione dei Giurati Semplici: il pattern dell'autenticazione anonima

Una delle sfide più complesse è stata quella di permettere a un *Simple Juror* di votare in modo sicuro senza obbligarlo a creare un account permanente. Risolvere questo problema solo con un token temporaneo sarebbe stato insicuro, poiché non avrebbe permesso di tracciare in modo affidabile se l'utente avesse già votato.

Per risolvere questa sfida, è stato implementato un pattern basato sull'**autenticazione anonima** offerta da Supabase. Questo approccio è un'applicazione diretta del Principio del Minimo Privilegio. Il flusso è il seguente:

1. **Creazione dell'account anonimo:** Nella pagina di accesso, l'utente seleziona l'opzione per partecipare come giurato semplice. L'applicazione gli chiede di inserire il proprio nome e cognome. Subito dopo, il client esegue una chiamata a Supabase per effettuare un **sign-in anonimo**.
2. **Autenticazione e Sessione JWT:** Supabase Auth crea un account utente a tutti gli effetti, ma marcato come "anonimo", e restituisce un JWT valido. Da questo momento, anche l'utente anonimo ha un `auth.uid()` univoco e una sessione autenticata. L'`AuthBloc` nel client entra in uno stato "autenticato ma anonimo".
3. **Accesso alla Home Page Limitata:** L'utente viene reindirizzato a una home page specifica per giurati semplici. L'`AuthGuard`, riconoscendo lo stato "anonimo", applica una logica di navigazione restrittiva: permette l'accesso solo a questa home page e alle pagine strettamente necessarie per la votazione, bloccando tutte le altre sezioni dell'app (es. impostazioni, gestione contest).

4. **Accesso alla Sessione di Voto:** Dalla sua home page, il giurato semplice ha due opzioni per accedere alla sessione di voto: può **scannerizzare un QR code** fornito dall'organizzatore o **inserire manualmente il token** della giuria.
5. **Validazione e Votazione:** Una volta inserito il token (tramite QR o manualmente), la RPC `juror_access_voting_as_simple_juror` lo valida. Se il token è corretto, l'utente viene reindirizzato alla procedura di voto, dove può esprimere le sue preferenze.
6. **Cancellazione dell'account al logout:** Al termine della procedura di voto, l'utente viene reindirizzato nuovamente alla sua home page limitata. Se a questo punto effettua il *logout*, viene invocata la Edge Function `user-delete-account`. Poiché l'utente possiede un JWT valido, è autorizzato a chiamare questa funzione, che elimina in modo sicuro e permanente l'account anonimo appena utilizzato, garantendo che nessun dato temporaneo rimanga nel sistema.

Questo pattern, oltre a garantire un'esperienza utente fluida e un alto livello di sicurezza, è stato progettato con un'importante considerazione sulla **scalabilità futura**. La gestione del giurato semplice tramite un vero account, seppur anonimo, pone le basi per future evoluzioni, come l'implementazione di una funzionalità che permetta all'utente di "reclamare" il proprio account anonimo e **convertirlo in un account permanente**.

I vantaggi principali di questo approccio sono:

- **Esperienza Utente Fluida:** L'utente non percepisce la creazione di un account, ma solo l'inserimento del proprio nome, seguito da un'interfaccia chiara per accedere alla votazione.
- **Sicurezza Robusta:** Ogni azione del giurato semplice è autenticata e legata a un `uid` univoco, prevenendo abusi.
- **Riutilizzo e Scalabilità dell'architettura:** Permette di riutilizzare la stessa logica di sicurezza (RLS, RPC, `AuthGuard`) sia per gli utenti normali che per quelli anonimi e apre la strada a future evoluzioni del sistema di account.

4.6 Progettazione e Implementazione degli Strumenti di Amministrazione

Oltre all'applicazione destinata agli utenti finali, un sistema software robusto richiede strumenti adeguati per la sua gestione, moderazione e manutenzione. Per Swift Contest, si è deciso di non sviluppare un'applicazione admin da zero, ma di utilizzare **Retool**, una piattaforma low-code specializzata nella creazione rapida di strumenti interni.

4.6.1 Esigenza di un Pannello di Amministrazione

Sebbene la dashboard di Supabase offra un accesso completo al database, essa è uno strumento pensato per gli sviluppatori, non per gli amministratori di sistema. La necessità di un pannello di amministrazione dedicato nasce quindi da requisiti operativi critici che, pur essendo tecnicamente possibili, **non sono gestibili in modo semplice, sicuro e rapido** tramite l'interfaccia standard di Supabase:

- **Gestione Utenti:** la capacità di visualizzare tutti gli utenti registrati, verificarne l'attività e, in caso di abuso, procedere con l'eliminazione dell'account.
- **Moderazione Contenuti:** la possibilità per un amministratore di ispezionare e, se necessario, rimuovere forzatamente un contest che viola i termini di servizio.

4.6.2 Scelta di Retool per lo Sviluppo Rapido

La scelta di Retool è stata strategica e motivata da:

- **Velocità di Sviluppo:** ha permesso di costruire un'interfaccia admin funzionale in poche ore, invece che in settimane, concentrando le risorse di sviluppo sull'applicazione principale.
- **Integrazione Nativa:** si integra nativamente con database PostgreSQL e API REST, permettendo di interfacciarsi facilmente con l'infrastruttura Supabase.
- **Sicurezza Configurabile:** offre meccanismi per gestire segreti (come le chiavi API) e configurare le chiamate in modo sicuro.


```
IF sessionuser <> 'retool' THEN -- Se il chiamante non l'utente 'retool' designato, --  
la funzione solleva un'eccezione e interrompe immediatamente l'esecuzione. RAISE EXCEPTION 'Pe  
This function can only be called by the admin user.'; END IF;
```

```
-- 2. ESECUZIONE DELLA LOGICA -- Se il controllo di sicurezza passa,  
la funzione procede -- con l'esecuzione della query per recuperare i dati. RETURN  
QUERY SELECT * FROM public.profiles;  
END;
```

;

Accesso tramite Edge Functions e API Key

Per le operazioni che richiedono un'orchestrazione complessa o l'interazione con servizi esterni al database, come l'invio di email o la manipolazione di file nello Storage, sono state implementate delle **Edge Functions**. Funzioni critiche come `admin-delete-account` e `admin-delete-contest` rientrano in questa categoria, poiché devono non solo modificare i record del database, ma anche eliminare tutti i file associati (immagini, file) dallo Storage per garantire una pulizia completa.

Queste funzioni sono protette da un meccanismo di autenticazione server-to-server per garantire che possano essere invocate solo da un client autorizzato come Retool:

1. Retool, nell'eseguire l'azione, invia una richiesta HTTP `POST` alla Edge Function, includendo un header personalizzato: `X-API-Key`.
2. Il valore di questa chiave è un segreto memorizzato sia in Retool che in Supabase.
3. La Edge Function, come primo passo, confronta la chiave ricevuta con quella memorizzata nei suoi segreti. Se non corrispondono, la richiesta viene immediatamente respinta con un errore 401 (Unauthorized).

Questo approccio garantisce che solo un'applicazione autorizzata possa invocare queste funzioni, proteggendo il sistema da accessi impropri.

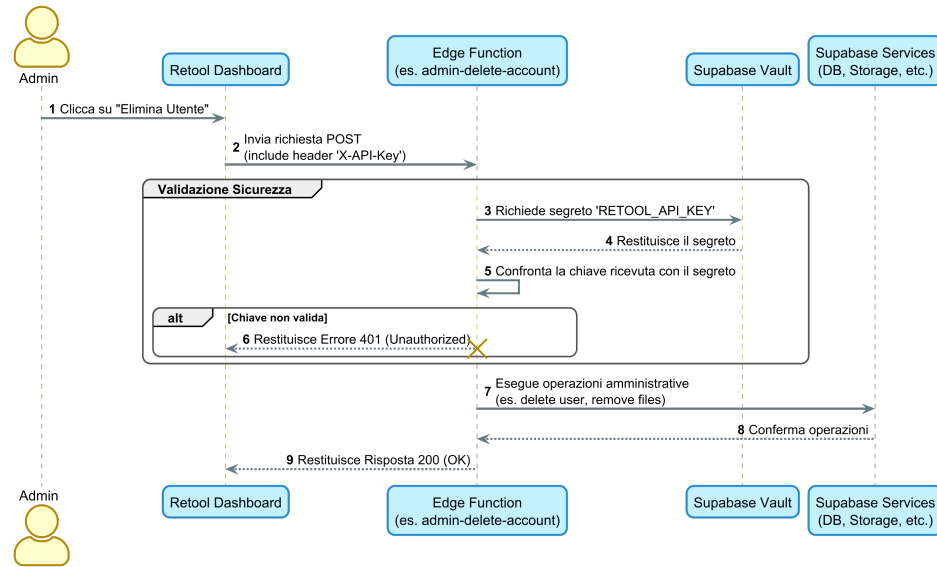


Figura 4.5

[language=TypeScript, caption=Validazione della chiave API all'interno di una Edge Function amministrativa., label=lst:apikey_validation] // EstrattodaunaEdgeFunctioncome'admin-delete-account/index.ts'

// ... (all'interno della funzione Deno.serve) try // — 1. API KEY AUTHORIZATION — // Questo pattern è usato per tutte le chiamate server-to-server da Retool.

// Legge la chiave API inviata dal client (Retool) nell'header. const apiKey = req.headers.get('X-API-Key');

// Recupera la chiave segreta memorizzata in modo sicuro nel Vault di Supabase. const secretKey = Deno.env.get('RETOOL_API_KEY');

// Primo controllo: verifica che il segreto sia stato configurato. if (!secretKey) throw new Error("RETOOL_API_KEY is not set in Supabase secrets.");

// Secondo controllo: confronta la chiave ricevuta con il segreto. // Se non corrispondono, la richiesta viene immediatamente respinta. if (apiKey !== secretKey) return new Response(JSON.stringify(error: "Unauthorized. Invalid API Key."), headers: ...corsHeaders, "Content-Type": "application/json" , status: 401, // 401 Unauthorized);

// — 2. PROSECUZIONE DELLA LOGICA — // Se la validazione ha successo, la funzione procede con // l'esecuzione delle operazioni amministrative.

```
// ... (logica per eliminare l'utente, i file, ecc.)  
catch (error) // ... (gestione degli errori)
```

4.6.4 Funzionalità Implementate nel Pannello

La dashboard sviluppata in Retool fornisce all'amministratore le seguenti funzionalità chiave:

- Una tabella per la visualizzazione e la ricerca di tutti gli utenti registrati.
- Una vista di dettaglio per un singolo utente, che mostra i contest a cui è associato e i suoi ruoli.
- Un pulsante “Elimina Utente”, che invoca in modo sicuro la Edge Function `admin-delete-account`.
- Una tabella di tutti i contest, con un pulsante “Elimina Contest” che invoca la Edge Function `admin-delete-contest`.

5.1 Sintesi del lavoro svolto e risultati ottenuti

Il progetto di tesi ha portato alla progettazione, implementazione e distribuzione di **Swift Contest**, una piattaforma software completa e funzionante per la gestione di concorsi, che raggiunge con successo tutti gli obiettivi prefissati in fase di analisi. È stato dimostrato come, attraverso l'integrazione di un ecosistema tecnologico moderno e accessibile, sia possibile costruire un sistema complesso, sicuro e scalabile, capace di risolvere le inefficienze e le criticità dei processi di gestione tradizionali.

I principali risultati ottenuti possono essere riassunti come segue:

- **Realizzazione di un'applicazione cross-platform unificata:** utilizzando Flutter, è stato sviluppato un unico client che offre un'esperienza utente coerente e funzionale sia su piattaforme web che mobile (Android), validando l'efficacia di un approccio a codebase singola per ridurre i tempi di sviluppo e la complessità di manutenzione.
- **Implementazione di un'architettura backend sicura e robusta:** è stato costruito un backend basato sul Principio del Minimo Privilegio, dove l'accesso ai dati è interamente mediato da un'API controllata, proteggendo l'integrità del sistema.

- **Creazione di un sistema di votazione affidabile e trasparente:** il meccanismo di *snapshot* delle sessioni di voto garantisce l'immutabilità dei dati e l'auditabilità del processo, risolvendo una delle principali criticità dei sistemi di valutazione manuali.
- **Validazione di un'infrastruttura efficiente e sostenibile:** l'architettura basata su servizi cloud (Supabase, Vercel) e l'uso mirato di risorse onerose come il *realtime* dimostrano la fattibilità di costruire soluzioni complesse mantenendo i costi operativi contenuti, in linea con i piani gratuiti o a basso costo offerti dai provider.

Il prototipo realizzato non è solo una dimostrazione tecnica, ma si presenta come un prodotto maturo e coeso, già utilizzabile in contesti reali a piccola e media scala.

5.2 Limiti e analisi critica

Un'analisi critica e onesta del lavoro svolto è fondamentale per comprenderne appieno il valore e i margini di miglioramento. Nonostante i risultati positivi, il progetto presenta alcuni limiti, intrinseci a un elaborato sviluppato in un contesto accademico con vincoli temporali definiti.

- **Logica di ranking semplificata:** l'attuale meccanismo di calcolo delle classifiche, basato sulla semplice somma dei punteggi dei campi *slider*, è funzionale ma basilare. Manca la possibilità di introdurre regole più complesse, come l'assegnazione di pesi diversi ai campi di voto o l'applicazione di algoritmi di scarto per i voti estremi (es. eliminare il voto più alto e più basso), limitandone l'applicabilità in contesti con regolamenti di valutazione più sofisticati.
- **Assenza di versioning dei form di voto:** la stabilità delle votazioni è garantita tramite snapshot, ma non esiste un vero sistema di versionamento per i *VotingForm*. Una volta che un form è stato usato, modificarlo potrebbe creare incongruenze se lo si volesse riutilizzare in futuro. Una gestione delle versioni permetterebbe agli organizzatori di creare e archiviare template di form riutilizzabili.

- **Osservabilità e monitoraggio limitati:** il sistema non integra strumenti avanzati di logging, metriche o tracing distribuito. In un ambiente di produzione su larga scala, l'assenza di questi strumenti renderebbe più complessa l'analisi delle performance, il debug di errori specifici e il monitoraggio proattivo della salute del sistema.
- **Test automatici non esaustivi:** sebbene la logica di business nei BLoC sia stata coperta da test di unità, il progetto beneficerebbe enormemente dall'aggiunta di una suite di test più completa, includendo test di integrazione per verificare l'interazione tra client e backend e test end-to-end per simulare i flussi utente completi.

5.3 Prospettive e sviluppi futuri

Il progetto attuale costituisce una solida base per numerosi e interessanti sviluppi futuri, che potrebbero trasformare Swift Contest da un prototipo avanzato a un prodotto commerciale completo e competitivo.

- **Ranking avanzati e configurabili:** lo sviluppo più immediato e di maggior valore sarebbe l'introduzione di un sistema che permetta agli organizzatori di definire regole di calcolo personalizzate per le classifiche, aumentando la flessibilità della piattaforma.
- **Supporto per sottomissioni multiple (Multi-Work):** estendere il modello dati per permettere ai partecipanti di inviare più di un *Work* all'interno dello stesso concorso è una funzionalità molto richiesta, specialmente in ambiti come la fotografia o la poesia.
- **Integrazione di notifiche push:** affiancare alle notifiche inbox un sistema di notifiche push (es. tramite Firebase Cloud Messaging o un altro provider) aumenterebbe significativamente il coinvolgimento e la reattività degli utenti, notificandoli istantaneamente di eventi importanti.

- **Hardening della sicurezza:** sebbene l'architettura sia sicura, potrebbe essere ulteriormente rafforzata con meccanismi di *rate limiting* sulle Edge Functions per prevenire abusi e attacchi di tipo DoS, e con policy RLS ancora più granulari.
- **Internazionalizzazione (i18n) e accessibilità (a11y):** la traduzione dell'interfaccia utente in più lingue e il miglioramento del supporto per le tecnologie assistive (es. screen reader) sono passi fondamentali per rendere l'applicazione accessibile a un pubblico globale e inclusivo.
- **Distribuzione ufficiale su App Store:** per completare la visione cross-platform, un passo futuro sarebbe quello di creare una versione per iOS e pubblicare entrambe le applicazioni mobile sugli store ufficiali (Google Play Store e Apple App Store), semplificandone la distribuzione e gli aggiornamenti.

5.4 Considerazioni finali

Swift Contest rappresenta un esempio concreto di come un'architettura software ben ponderata, unita a un ecosistema tecnologico moderno e accessibile, possa portare alla creazione di un sistema complesso, sicuro e di valore. La scelta di combinare la flessibilità di Flutter con la potenza e la sicurezza di Supabase si è rivelata vincente, permettendo di raggiungere un ottimo equilibrio tra rapidità di sviluppo, ricchezza funzionale e sostenibilità dei costi.

Il progetto non si esaurisce con la presente tesi; esso costituisce una base di partenza robusta e scalabile per future iterazioni e, potenzialmente, per applicazioni in contesti reali. Il lavoro svolto conferma la validità dell'approccio architetturale adottato e apre prospettive interessanti nel campo delle piattaforme collaborative e dei sistemi di votazione digitale, dimostrando che è possibile realizzare prototipi di alta qualità anche con le risorse a disposizione di uno studente o di un piccolo team di sviluppo.

Ringraziamenti

Ringraziamenti qui...